EBOOK THỰC HÀNH · CẬP NHẬT 10/2026

Hướng dẫn sử dụng

GITHUB

TỪ A ĐẾN Z

toàn tập

Dành cho người mới bắt đầu, kể cả người chưa từng viết code.
Từng bước bấm, bài tập thực hành, video chọn lọc có QR,
đối chiếu tài liệu chính thức docs.github.com và github.blog.

REPO · COMMIT · PULL REQUEST · ACTIONS · PAGES · COPILOT

by

Minh Đỗ

AI EDUCATION CREATOR

facebook.com/dotanminh

Tài liệu độc lập · không liên kết, không được GitHub bảo trợ · GitHub và Copilot là nhãn hiệu của GitHub, Inc.

TRƯỚC KHI BẮT ĐẦU

Lời mở đầu

GitHub nghe có vẻ là chỗ của lập trình viên. Thật ra, bất kỳ ai cần lưu trữ tài liệu có lịch sử, làm việc nhóm hay đăng một trang web nhỏ đều dùng được.

Sách viết cho người mới hoàn toàn: sinh viên, giáo viên, người làm nội dung, chủ shop muốn có trang giới thiệu. Bạn không cần biết lập trình. Phần lớn các chương làm được ngay trên trình duyệt; phần Git trên máy, GitHub Desktop, CLI dành cho lúc bạn đã quen.

i. Cách đọc cuốn sách

- **Đọc theo thứ tự** từ chương 1 đến 15 nếu bạn mới hoàn toàn: đó là phần "làm trên web".
- **Chương 16–19**: đưa GitHub về máy tính (Git, GitHub Desktop, GitHub CLI, .gitignore).
- **Chương 20–30**: các tính năng mở rộng (Releases, Pages, Actions, Codespaces, Copilot, bảo mật, tổ chức, gói cước).
- **Chương 31–32 và phụ lục**: thói quen tốt, sự cố, FAQ, thuật ngữ, checklist, bảng tra nhanh.

ii. Ký hiệu trong sách

Ký hiệu	Ý nghĩa
THEO DOCS	Thông tin lấy trực tiếp từ tài liệu chính thức docs.github.com, có số trích dẫn [n] dẫn tới danh mục nguồn cuối sách.
THAY ĐỔI 2026	Tính năng hoặc chính sách mới đổi trong năm 2026, bạn có thể thấy khác với video hay bài viết cũ.
⚠ CHƯA CHẮC	Chưa xác minh được đầy đủ từ nguồn chính thức, hoặc dễ thay đổi. Hãy kiểm tra lại trên trang GitHub trước khi làm theo.
BÀI TẬP	Bài tập thực hành, làm xong mới chuyển chương.
xem từ mm:ss đến mm:ss	Thẻ video: quét QR bằng camera điện thoại, video mở đúng giây bắt đầu; xem tới mốc kết thúc là đủ.

iii. Nguyên tắc biên soạn

- Thông tin về tính năng, giá, giới hạn được đối chiếu với docs.github.com, github.com/pricing và github.blog tại thời điểm 17/10/2026 (giờ Việt Nam, UTC+7).
- Ảnh giao diện đều là ảnh chính thức trong GitHub Docs, có ghi "Nguồn:" bên dưới. Sách không vẽ giả giao diện. Giao diện thật có thể đổi nhẹ theo thời gian.
- Video của kênh uy tín (ưu tiên kênh chính thức GitHub), lượt xem đo ngày 17/10/2026. Video tiếng Anh; có thể bật phụ đề tự dịch.
- Mọi ví dụ về mật khẩu, token, khóa đều là chỗ trống như `<YOUR_TOKEN>`. Đừng bao giờ dán khóa thật vào nơi công khai.

> MỘT LỜI KHUYÊN
>
> Mở GitHub ở tab bên cạnh, làm theo ngay khi đọc. Đến cuối sách, bạn sẽ có: một tài khoản bảo mật tốt, hồ sơ cá nhân đẹp, một repo có README, vài pull request đã merge, một trang web trên GitHub Pages và một workflow Actions chạy tự động.

NỘI DUNG

Mục lục

01 GitHub là gì? 5
Git, GitHub và các khái niệm cốt lõi

02 Tài khoản và hồ sơ cá nhân 7
Đăng ký, các loại tài khoản, profile README

03 Bảo mật tài khoản: 2FA và passkey 9
Xác thực hai lớp, passkey, mã khôi phục

04 Repository đầu tiên 11
Tạo repo, công khai hay riêng tư, cộng tác viên

05 README và Markdown 13
Viết tài liệu đẹp bằng vài ký hiệu đơn giản

06 Làm việc với file trên web 15
Tạo, tải lên, sửa file ngay trên trình duyệt

07 Commit và lịch sử thay đổi 17
Mỗi commit là một bản lưu có tên, có thời gian, có lý do

08 Branch: làm việc song song 19
Tạo, chuyển, đổi tên, xóa nhánh

09 Pull request 21
Đề xuất thay đổi, thảo luận và gộp code

10 Code review và merge 23
Duyệt thay đổi, gợi ý sửa, chọn cách gộp

11 Xử lý xung đột (merge conflict) 25
Khi hai người sửa cùng một chỗ

12 Issues, labels và milestones 27
Ghi việc, báo lỗi, phân loại và theo dõi tiến độ

13 GitHub Projects 29
Bảng kế hoạch dạng bảng, kanban và lộ trình

14 GitHub Discussions 31
Diễn đàn hỏi đáp cho cộng đồng của repo

15 Fork, Star, Watch và đóng góp mã nguồn mở 32
Theo dõi dự án hay và gửi đóng góp đầu tiên

16 Git trên máy tính 34
Cài Git, cấu hình, clone, push, pull, đăng nhập

17 GitHub Desktop 37
Dùng Git bằng giao diện, không cần gõ lệnh

18 GitHub CLI (gh) 39
Điều khiển GitHub từ cửa sổ dòng lệnh

19 .gitignore và file không nên đưa lên 41
Giữ repo sạch, không lộ thông tin

20 License, Releases và Tags — 43
Cho phép người khác dùng, đóng gói phiên bản

21 GitHub Pages: trang web miễn phí — 45
Đăng CV, portfolio, blog, trang dự án

22 GitHub Actions cơ bản — 47
Tự động hóa: kiểm thử, build, deploy, dọn issue

23 GitHub Codespaces — 50
Môi trường lập trình trên mây, mở bằng trình duyệt

24 GitHub Copilot — 52
Gói cước, Chat, agent, code review, CLI và Copilot app

25 Bảo mật repository — 57
Dependabot, secret scanning, push protection, code scanning

26 Gists — 59
Chia sẻ nhanh đoạn văn bản, ghi chú, đoạn code

27 Organizations và Teams — 60
Làm việc nhóm, lớp học, công ty

28 Thông báo và bảng điều khiển — 61
Inbox, mức theo dõi, dashboard mới

29 GitHub Mobile — 63
GitHub trong túi: iOS và Android

30 Gói dịch vụ và chi phí — 64
Free, Pro, Team, Enterprise và các khoản tính theo mức dùng

31 Thói quen tốt — 66
15 nguyên tắc giúp repo gọn, an toàn, dễ cộng tác

32 Xử lý sự cố thường gặp — 67
Đọc thông báo lỗi, sửa từng bước

· Câu hỏi thường gặp — 68
25 câu hỏi của người mới

· Thuật ngữ — 70
60 từ khóa Anh–Việt

· Checklist tự đánh giá — 71
Đánh dấu khi đã làm được

· Bảng tra nhanh lệnh và prompt — 72
Git, gh, Copilot CLI, Markdown, prompt mẫu

· Danh mục video tham khảo — 74
16 đoạn video có QR, đã kiểm tra

· Nguồn tham khảo — 75
docs.github.com, github.blog, github.com/pricing

01 CHƯƠNG 1

GitHub là gì?

Hiểu đúng GitHub trong 10 phút: nó lưu gì, khác Git ở đâu, và vì sao cả thế giới dùng nó để làm việc chung.

1.1 Git và GitHub: hai thứ khác nhau

Git là phần mềm quản lý phiên bản chạy trên máy bạn: nó ghi lại lịch sử thay đổi của file, cho phép quay lại bản cũ và làm nhiều hướng song song. **GitHub** là nền tảng trên mạng để lưu các kho Git, chia sẻ và cộng tác: bạn có giao diện web, pull request, issue, Actions, Pages, Copilot...[1][2]

	Git	GitHub
Là gì	Phần mềm cài trên máy, mã nguồn mở	Dịch vụ web (github.com), có ứng dụng Desktop, CLI, Mobile
Dùng để	Lưu lịch sử, tạo nhánh, gộp thay đổi	Lưu kho trực tuyến, cộng tác, review, tự động hóa, đăng web
Cần mạng?	Không, chạy offline được	Có
Bắt buộc?	Không bắt buộc nếu chỉ dùng web	Cần tài khoản để tạo repo, đóng góp

1.2 Bảy khái niệm phải nhớ

Thuật ngữ	Hiểu nôm na
Repository (repo)	Một "thư mục dự án" trên GitHub, chứa file và toàn bộ lịch sử thay đổi.
Commit	Một lần "chụp ảnh" các thay đổi, kèm lời nhắn mô tả và tên người thực hiện.
Branch (nhánh)	Một đường làm việc song song; nhánh chính thường tên `main`.
Pull request (PR)	Đề nghị gộp thay đổi từ nhánh này vào nhánh khác, kèm thảo luận và review.
Merge	Gộp thay đổi của nhánh vào nhánh đích.
Issue	Một "phiếu việc": báo lỗi, đề xuất ý tưởng, ghi việc cần làm.
Fork	Bản sao một repo của người khác về tài khoản bạn để tự do sửa và đề xuất lại.

Định nghĩa đầy đủ: GitHub glossary[3] và phần Thuật ngữ cuối sách.

1.3 Ai dùng GitHub, dùng để làm gì?

- Lập trình viên: lưu code, review chéo, kiểm thử tự động.
- Người không code: viết tài liệu bằng Markdown có lịch sử, quản lý công việc nhóm bằng Issues và Projects, đăng blog, CV, trang giới thiệu bằng GitHub Pages.
- Giáo viên, sinh viên: nộp bài, làm đồ án nhóm, xây portfolio. Sinh viên đã xác minh được dùng Copilot Student miễn phí[4].



▶ VIDEO THAM KHẢO · TIẾNG ANH · QUÉT QR

Git Explained in 100 Seconds

xem từ 00:00 đến 01:57 kênh **Fireship** · 841.845 lượt xem (17/10/2026) · đăng 02/03/2020

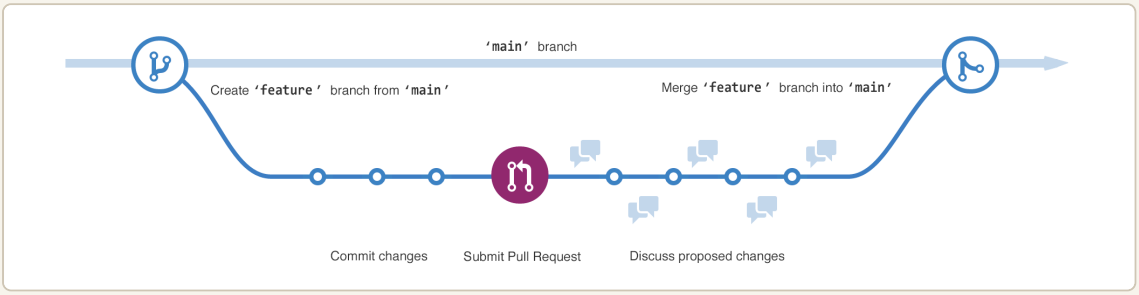
Đoạn: toàn bộ video (6 chương ngắn). Hai phút: repo, stage, commit, nhánh, merge.

youtu.be/hwP7WQkmECE?t=0

1.4 Luồng làm việc cơ bản: GitHub flow

GitHub mô tả một quy trình nhẹ gọi là **GitHub flow**[5]. Bạn sẽ gặp lại nó ở hầu hết các chương:

1. Tạo một **nhánh** mới từ `main` để làm việc, không sửa thẳng vào nhánh chính.
2. Thực hiện thay đổi và tạo **commit** (mỗi commit một ý rõ ràng).
3. Mở **pull request** để mọi người xem, góp ý.
4. Trao đổi, sửa theo **review**.
5. **Merge** pull request vào `main`.
6. Xóa nhánh đã dùng xong cho gọn.



Nhánh "feature" tách ra từ "main", sau đó được merge trở lại main.
Nguồn: ảnh chính thức GitHub Docs, trang "Hello World" [6]

1.5 Bạn cần chuẩn bị gì?

- Một địa chỉ email bạn dùng thường xuyên (để xác minh và khôi phục tài khoản).
- Trình duyệt hiện đại (Chrome, Edge, Firefox, Safari).
- Điện thoại có ứng dụng xác thực (TOTP) hoặc passkey (chương 3).
- Không bắt buộc: máy tính Windows hoặc macOS để cài GitHub Desktop (chương 17), Git (chương 16).

▶ VIDEO THAM KHẢO · TIẾNG ANH · QUÉT QR

A brief introduction to Git for beginners | GitHub

xem từ 00:44 đến 03:06 kênh **GitHub** · 2.225.973 lượt xem (17/10/2026) · đăng 26/05/2024

Đoạn: "What is Git?" + "Basic Git Concepts". Video chính thức của GitHub: Git là gì và các khái niệm cơ bản (repo, commit, branch).

youtu.be/r8jQ9hVA2qs?t=44

BÀI TẬP · LÀM QUEN

1. Mở `github.com/explore`, xem các repo nổi bật.
2. Mở một repo bất kỳ, tìm: danh sách file, nút **Code**, tab **Issues**, tab **Pull requests**, số sao (Star).
3. Ghi lại 3 thuật ngữ bạn chưa hiểu, tra ở phần Thuật ngữ cuối sách.

Kết quả cần đạt: Bạn nhận ra được các phần chính của một trang repo.

02 CHƯƠNG 2

Tài khoản và hồ sơ cá nhân

Tài khoản cá nhân là "danh tính" của bạn trên GitHub. Chọn tên đẹp, điền hồ sơ gọn gàng ngay từ đầu.

2.1 Ba loại tài khoản

Loại	Dùng khi
Personal account (cá nhân)	Mỗi người một tài khoản. Mọi hoạt động trên GitHub đều gắn với tài khoản cá nhân.
Organization (tổ chức)	Tài khoản dùng chung cho nhóm, công ty, lớp học; có nhiều thành viên và phân quyền (chương 27).
Enterprise	Dành cho doanh nghiệp lớn quản lý nhiều tổ chức (GitHub Enterprise Cloud).

THEO DOCS Theo "Types of GitHub accounts"[7]: mỗi người dùng luôn đăng nhập bằng tài khoản cá nhân; tổ chức là nơi nhiều tài khoản cá nhân cộng tác[8].

2.2 Đăng ký tài khoản

1. Vào `github.com/signup`.
2. Nhập email, tạo mật khẩu mạnh, chọn **username** (tên này xuất hiện trong mọi đường dẫn: `github.com/<username>`).
3. Hoàn tất bước xác minh (giải đố) và nhập mã gửi về email.
4. Chọn gói **Free** (đủ cho mọi bài tập trong sách).

THEO DOCS Quy trình chi tiết: "Creating an account on GitHub"[9]. Màn hình đăng ký có thể đổi câu chữ theo thời gian.

> MẸO CHỌN USERNAME
>
> - Ngắn, dễ đọc, không dấu, giống tên bạn dùng ở nơi khác (ví dụ `minhdo-dev`).
> - Tránh số năm sinh, ký tự khó đọc. Đổi username sau này sẽ làm hỏng các đường dẫn cũ.
> - Dùng email cá nhân lâu dài, không dùng email công ty/trường sắp hết hạn.

2.3 Hoàn thiện hồ sơ (profile)

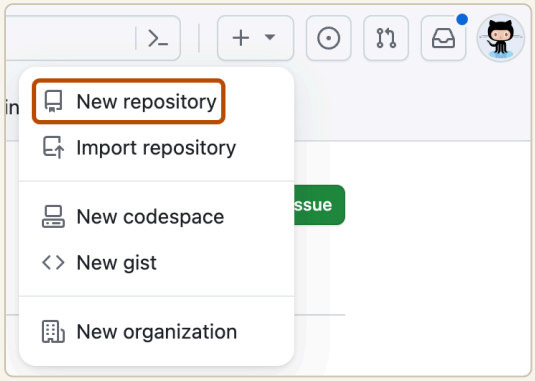
Bấm ảnh đại diện góc phải trên, chọn **Your profile** rồi **Edit profile**. Bạn có thể thêm ảnh, tên hiển thị, giới thiệu ngắn (bio), nơi làm việc, website, mạng xã hội, và ghim (pin) các repo tiêu biểu[10].

- **Ảnh đại diện**: ảnh rõ mặt hoặc logo, nền đơn giản.
- **Bio**: một câu "bạn là ai, làm gì".
- **Pinned repositories**: tối đa vài repo bạn muốn người khác xem đầu tiên.
- **Contribution graph**: các ô xanh thể hiện hoạt động theo ngày.

2.4 Profile README: "trang bìa" của bạn

Nếu bạn tạo một repo **công khai** có tên **trùng đúng username** và có file `README.md` ở thư mục gốc, nội dung README đó sẽ hiện ngay đầu trang hồ sơ[11].

1. Bấm dấu **+** góc phải trên, chọn **New repository**.
2. Ở ô **Repository name**, gõ đúng username của bạn. GitHub sẽ báo đây là repo đặc biệt.
3. Chọn **Public**, tích **Add a README file**, bấm **Create repository**.
4. Bấm biểu tượng bút chì trên README để sửa, rồi commit.



Menu dấu "+" góc phải trên, mục "New repository".
Nguồn: ảnh chính thức GitHub Docs, trang "Quickstart for repositories" [12]

MẪU README HỒ SƠ — MARKDOWN

```
## Xin chào, mình là Minh 👋
- Mình đang học: GitHub, Markdown, GitHub Pages
- Dự án tiêu biểu: [Trang giới thiệu](https://<username>.github.io)
- Liên hệ: facebook.com/<ten-cua-ban>
```

Lưu ý từ docs: nếu repo trùng tên được tạo trước tháng 7/2020, README không tự hiện; vào repo bấm **Share to profile**[11].

BÀI TẬP · HỒ SƠ CỦA TÔI

1. Thêm ảnh đại diện và bio.
2. Tạo repo trùng username, viết README 4–6 dòng theo mẫu.
3. Mở `github.com/<username>` kiểm tra README đã hiện.

Kết quả cần đạt: Trang hồ sơ có ảnh, bio và phần giới thiệu.

03 CHƯƠNG 3

Bảo mật tài khoản: 2FA và passkey

Tài khoản GitHub giữ code, khóa, đôi khi cả trang web của bạn. Bật bảo mật ngay hôm nay, mất 10 phút.

3.1 Xác thực hai lớp (2FA) là gì?

2FA yêu cầu thêm một bước ngoài mật khẩu khi đăng nhập: mã từ ứng dụng trên điện thoại (TOTP), tin nhắn SMS, khóa bảo mật, passkey, hoặc xác nhận trên GitHub Mobile[13].

BẮT BUỘC VỚI NGƯỜI ĐÓNG GÓP CODE

THEO DOCS Từ tháng 3/2023, GitHub yêu cầu người đóng góp code trên GitHub.com bật 2FA theo từng đợt. Nếu được chọn, bạn có 45 ngày để bật, thêm 7 ngày ân hạn; quá hạn sẽ không vào được GitHub.com cho tới khi bật 2FA[14].

3.2 Phương thức nào nên dùng?

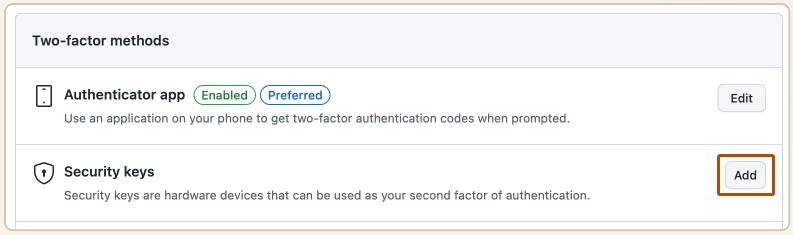
Phương thức	Ưu điểm	Ghi chú
Ứng dụng TOTP	GitHub khuyên dùng làm phương thức chính	Google Authenticator, Microsoft Authenticator, 1Password…
Passkey	Chống lừa đảo tốt nhất, đăng nhập một bước	Lưu trên điện thoại, Windows Hello, trình quản lý mật khẩu
Khóa bảo mật	Rất an toàn (FIDO2, WebAuthn)	Cần thiết bị vật lý
GitHub Mobile	Bấm xác nhận trên app	Thêm sau khi đã có TOTP hoặc SMS
SMS	Dễ dùng	Kém an toàn nhất, có thể bị lừa lấy mã

THEO DOCS GitHub khuyên dùng TOTP làm phương thức chính và thêm passkey hoặc khóa bảo mật làm dự phòng[14].

3.3 Bật 2FA bằng ứng dụng TOTP

1. Ảnh đại diện › **Settings** › mục **Password and authentication**.
2. Bấm **Enable two-factor authentication**.
3. Mở ứng dụng xác thực trên điện thoại, quét mã QR trên màn hình, nhập mã 6 số.
4. **Tải về mã khôi phục (recovery codes)** bằng nút **Download**, cất ở nơi an toàn, rồi bấm **I have saved my recovery codes**.

THEO DOCS Các bước theo “Configuring two-factor authentication”[15]; tên nút có thể đổi nhẹ.



Phần "Two-factor methods" trong cài đặt 2FA, nơi thêm khóa bảo mật hoặc phương thức khác.
Nguồn: ảnh chính thức GitHub Docs, trang "Configuring two-factor authentication" [15]

3.4 Passkey: đăng nhập không cần mật khẩu

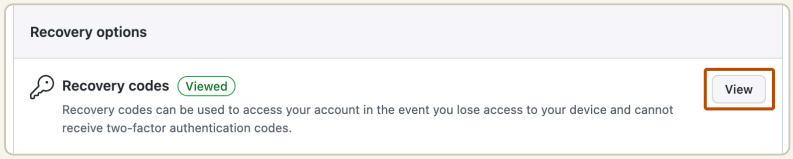
Passkey là cặp khóa mã hóa lưu trong thiết bị bạn kiểm soát (điện thoại, Windows Hello, khóa FIDO2, trình quản lý mật khẩu). Nếu bạn đã bật 2FA, passkey thỏa cả mật khẩu và 2FA, đăng nhập chỉ một bước.

Passkey gắn với tên miền `github.com`, nên trang giả mạo không lừa được[16].

1. **Settings** › **Password and authentication** › phần **Passkeys**.
2. Bấm **Add a passkey**, xác nhận bằng mật khẩu hoặc 2FA.
3. Làm theo hộp thoại của trình duyệt (vân tay, khuôn mặt, PIN).

THEO DOCS Quản lý passkey: "Managing your passkeys"[17]. Docs khuyên vẫn giữ một phương thức 2FA dự phòng (TOTP hoặc SMS) để tránh bị khóa ngoài[13].

3.5 Mã khôi phục và phương án dự phòng



Nút "View" để xem lại mã khôi phục trong phần "Recovery options".
Nguồn: ảnh chính thức GitHub Docs, trang "Configuring two-factor authentication recovery methods" [18]

- In hoặc lưu mã khôi phục vào trình quản lý mật khẩu. Mỗi mã chỉ dùng một lần.
- Thêm ít nhất hai phương thức (ví dụ TOTP và passkey).
- Mất hết phương thức và mã khôi phục: xem quy trình "Recovering your account if you lose your 2FA credentials"[19]. Quá trình có thể mất nhiều ngày.

KHÔNG BAO GIỜ

- Chia sẻ mã 2FA, mã khôi phục hay mật khẩu cho bất kỳ ai, kể cả người tự xưng là "nhân viên GitHub".
- Lưu mã khôi phục trong một repo, kể cả repo private.

BÀI TẬP · KHÓA CHẶT TÀI KHOẢN

1. Bật 2FA bằng ứng dụng TOTP.
2. Tải mã khôi phục, cất vào 2 nơi khác nhau.
3. Thêm một passkey.
4. Đăng xuất, đăng nhập lại bằng passkey để thử.

Kết quả cần đạt: Tài khoản có 2FA, có passkey, có mã khôi phục được cất giữ.

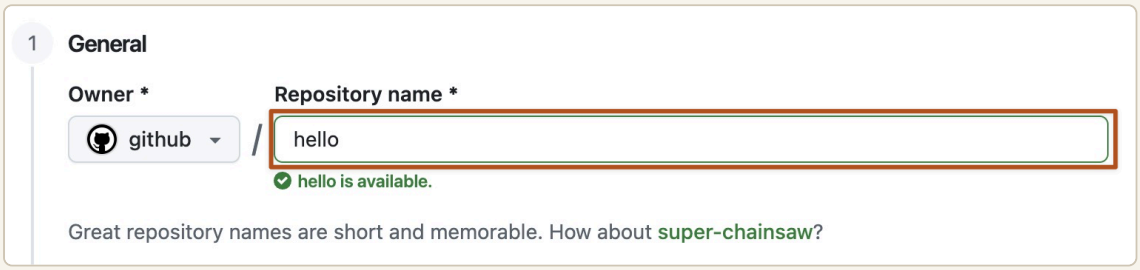
04 CHƯƠNG 4

Repository đầu tiên

Repository là nơi mọi thứ bắt đầu. Chương này giúp bạn tạo repo đúng cách và hiểu các lựa chọn khi tạo.

4.1 Tạo repository mới

1. Bấm **+** góc phải trên › **New repository**.
2. Chọn **Owner** (bạn hoặc một tổ chức) và gõ **Repository name**, ví dụ `hello-world`.
3. Viết **Description** ngắn.
4. Chọn **Public** hoặc **Private**.
5. Tích **Add a README file**; tùy chọn thêm `.gitignore` và license.
6. Bấm **Create repository**.



Bước đặt tên repository: ô "Repository name" với tên "hello-world".
Nguồn: ảnh chính thức GitHub Docs, trang "Quickstart for repositories" [12]

THEO DOCS Theo "Quickstart for repositories"[12].

4.2 Public hay Private?

	Public (công khai)	Private (riêng tư)
Ai xem được	Mọi người trên internet	Chỉ bạn và người được mời
Phù hợp	Mã nguồn mở, portfolio, tài liệu chia sẻ, GitHub Pages miễn phí	Dự án cá nhân, bài tập chưa nộp, dữ liệu nội bộ
Lưu ý	Đừng để mật khẩu, khóa API, dữ liệu cá nhân	Vẫn không nên lưu khóa thật; một số tính năng nâng cao cần gói trả phí

THEO DOCS Đổi chế độ hiển thị ở **Settings** › **Danger Zone** › **Change visibility**[20]. Repo cá nhân Free: không giới hạn repo public và private[21].

4.3 Một repo gồm những gì?

Thành phần	Vai trò
Tab Code	Danh sách file, thư mục, README hiển thị bên dưới.
Tab Issues	Báo lỗi, ý tưởng, việc cần làm (chương 12).
Tab Pull requests	Đề xuất thay đổi đang chờ review (chương 9).
Tab Actions	Workflow tự động (chương 22).
Tab Projects, Wiki, Security, Insights	Kế hoạch, tài liệu, cảnh báo bảo mật, thống kê.
Tab Settings	Đổi tên, quyền truy cập, nhánh mặc định, Pages, xóa repo.

Tên tab có thể khác chút tùy repo và gói; tab Settings chỉ hiện khi bạn có quyền quản trị[22].

4.4 Mời cộng tác viên

Với repo cá nhân, vào **Settings** › **Collaborators** › **Add people**, nhập username hoặc email. Người được mời phải chấp nhận lời mời qua email hoặc thông báo[23]. Cộng tác viên của repo cá nhân có quyền ghi (push) vào repo.

4.5 Giới hạn kích thước cần biết

- Tải file qua trình duyệt: tối đa **25 MiB** mỗi file.
- Từ 50 MiB Git sẽ cảnh báo; GitHub **chặn** file lớn hơn **100 MiB**.
- File lớn (video, dữ liệu): cân nhắc Git LFS hoặc đính kèm vào Releases.

THEO DOCS "About large files on GitHub"[24].

ĐẶT TÊN REPO CHO DỄ DÙNG

- Chữ thường, không dấu, nối bằng gạch ngang: `ban-tin-lop-12a`.
- Tên mô tả nội dung, tránh `test1`, `new-folder`.
- Luôn có README để người khác hiểu repo dùng làm gì.

BÀI TẬP · REPO ĐẦU TIÊN

1. Tạo repo public `hello-world` có README.
2. Thêm mô tả (Description) và vài chủ đề (Topics) ở ô ⚙ bên phải trang repo.
3. Mời một người bạn làm cộng tác viên (nếu có).

Kết quả cần đạt: Repo `hello-world` hiển thị README và mô tả.

05 CHƯƠNG 5

README và Markdown

Markdown là "ngôn ngữ" định dạng văn bản của GitHub: README, issue, bình luận, Wiki đều dùng nó.

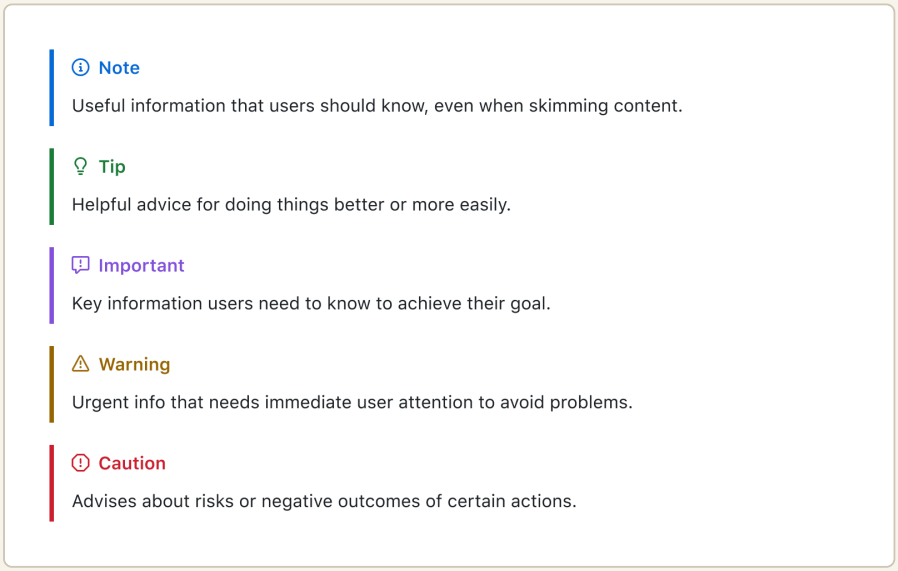
5.1 README là gì?

README là file đầu tiên người khác đọc khi vào repo: dự án làm gì, cách dùng, ai duy trì. GitHub tự hiển thị README ở trang chính của repo, và tự tạo mục lục từ các tiêu đề[25].

5.2 Cú pháp Markdown cơ bản

Bạn gõ	Kết quả
`# Tiêu đề 1`, `## Tiêu đề 2`, `### Tiêu đề 3`	Các cấp tiêu đề
`**đậm**`, `*nghiêng*`, `~~gạch~~`	**đậm**, *nghiêng*, ~~gạch~~
`- mục` hoặc `1. mục`	Danh sách chấm, danh sách số
`[chữ](https://...)`	Liên kết
`![mô tả ảnh](duong-dan-anh.png)`	Chèn ảnh
`` `lệnh` `` và khối ``` ... ```	Mã trong dòng, khối mã
`> trích dẫn`	Đoạn trích dẫn
`- [ ] việc`, `- [x] xong`	Danh sách việc có ô tích
`@username`, `#12`	Nhắc tên người, liên kết tới issue/PR số 12
`> [!NOTE]`, `[!TIP]`, `[!WARNING]`...	Hộp cảnh báo (alerts) có màu

THEO DOCS Theo "Basic writing and formatting syntax"[26] và "Autolinked references and URLs"[27].



Năm loại hộp cảnh báo Markdown: Note, Tip, Important, Warning, Caution.
Nguồn: ảnh chính thức GitHub Docs, trang "Basic writing and formatting syntax" [26]

5.3 Mẫu README cho người mới

README.MD MẪU

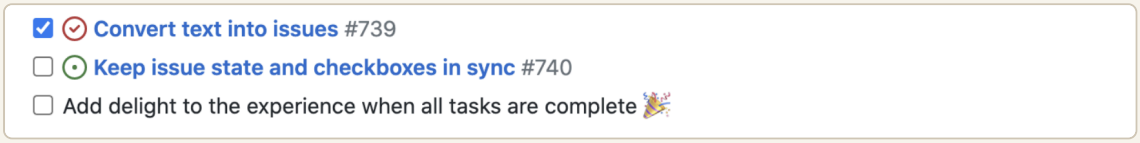
```
# Tên dự án

Một câu giới thiệu dự án làm gì, cho ai.

## Tính năng
- Tính năng 1
- Tính năng 2

## Cách dùng
1. Bước 1
2. Bước 2

## Liên hệ
Tạo issue hoặc nhắn @<username>.
```



Danh sách việc (task list) sau khi hiển thị: ô tích và liên kết tới issue được tạo tự động.

Nguồn: ảnh chính thức GitHub Docs, trang "Basic writing and formatting syntax" [26]

5.4 Mẹo viết Markdown

- Dùng tab **Preview** khi sửa file trên web để xem trước.
- Để xuống dòng trong cùng đoạn, để trống một dòng giữa hai đoạn.
- Bảng: dùng `| cột 1 | cột 2 |` và dòng `| --- | --- |` bên dưới.
- Kéo thả ảnh vào ô bình luận issue/PR, GitHub tự tải lên và chèn Markdown.

PROMPT NHỜ AI (COPILOT CHAT, GROK…) VIẾT README — COPILOT CHAT

Hãy viết giúp tôi một README.md bằng tiếng Việt cho repo <tên-repo>: dự án làm <mô tả>. Gồm: giới thiệu 2 câu, tính năng, cách dùng từng bước, cách đóng góp, giấy phép. Dùng Markdown của GitHub, có một hộp [!TIP].

BÀI TẬP · README ĐẸP

1. Mở repo `hello-world`, sửa README theo mẫu 5.3.
2. Thêm một bảng 2 cột và một hộp `> [!TIP]`.
3. Thêm danh sách việc 3 mục, tích 1 mục.
4. Xem Preview rồi commit.

Kết quả cần đạt: README có tiêu đề, danh sách, bảng, hộp TIP và task list.

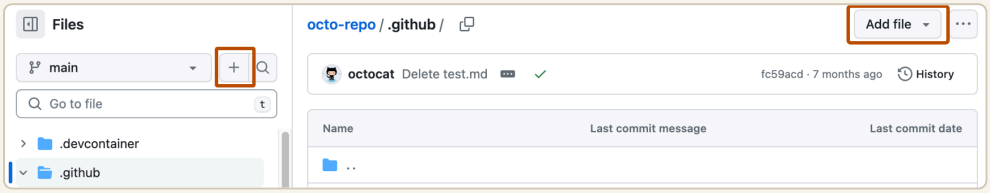
06 CHƯƠNG 6

Làm việc với file trên web

Không cần cài gì: bạn tạo file mới, kéo thả file từ máy, sửa nội dung và lưu thay đổi ngay trên github.com.

6.1 Tạo file mới

1. Mở repo, bấm **Add file** › **Create new file** (hoặc dấu **+**).
2. Gõ tên file, ví dụ `gioi-thieu.md`. Gõ `ten-thu-muc/` trước tên để tạo thư mục.
3. Viết nội dung, xem tab **Preview**.
4. Bấm **Commit changes...**, viết lời nhắn commit, chọn commit thẳng vào nhánh hiện tại hoặc tạo nhánh mới, rồi xác nhận.



Nút "Add file" và dấu "+" trên trang chính của repo.
Nguồn: ảnh chính thức GitHub Docs, trang "Creating new files" [28]

THEO DOCS "Creating new files"[28].

6.2 Tải file từ máy lên

1. Bấm **Add file** › **Upload files**, hoặc kéo thả file/thư mục vào trình duyệt.
2. Chờ tải xong, viết lời nhắn commit, bấm **Commit changes**.

Giới hạn khi tải qua trình duyệt

Kích thước mỗi file	tối đa 25 MiB (qua dòng lệnh: tối đa 100 MiB)
Số file mỗi lần	tối đa 100 file

THEO DOCS "Adding a file to a repository"[29].

6.3 Sửa file

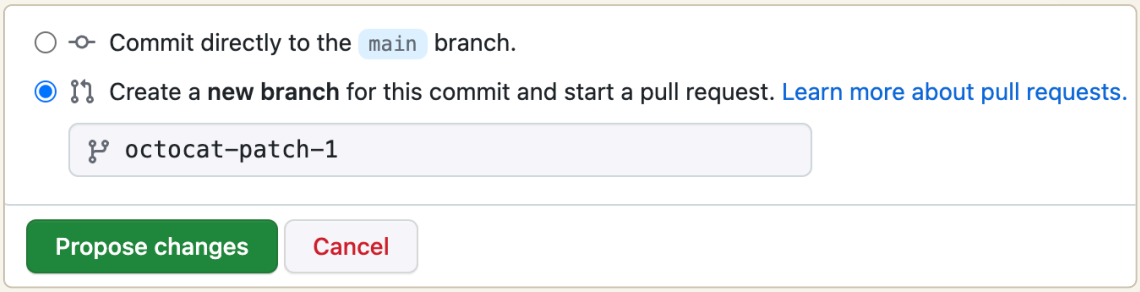
1. Mở file, bấm biểu tượng **bút chì** (Edit) góc phải trên của file.
2. Sửa nội dung; với file Markdown dùng tab **Preview** để xem trước.
3. Bấm **Commit changes...** và chọn nơi lưu.



Nút bút chì để sửa file trên GitHub.
Nguồn: ảnh chính thức GitHub Docs, trang "Editing files" [30]

6.4 Commit thẳng hay tạo nhánh mới?

Khi lưu, GitHub hỏi bạn commit thẳng vào nhánh đang mở (thường là `main`) hay **tạo nhánh mới và mở pull request**. Với repo cá nhân, commit thẳng cho nhanh. Khi làm nhóm, hãy chọn nhánh mới để người khác review trước[30].



Hộp thoại commit: chọn commit thẳng vào main hoặc tạo nhánh mới.
Nguồn: ảnh chính thức GitHub Docs, trang "Creating new files" [28]

6.5 Đổi tên, di chuyển, xóa file

- **Đổi tên / chuyển thư mục**: mở chế độ sửa, sửa ô tên file; thêm `thu-muc/` phía trước để chuyển, `../` để lùi một cấp.
- **Xóa**: mở file, bấm nút **...** (More) › **Delete file**, rồi commit.
- Mọi thao tác đều tạo commit, nên luôn có thể xem lại hoặc khôi phục từ lịch sử.

⚠ CHƯA CHẮC Vị trí menu xóa, đổi tên có thể khác chút theo giao diện; xem "Editing files"[30].

6.6 Phím tắt hữu ích trên github.com

Phím	Tác dụng
`t`	Tìm nhanh file trong repo
`.` (dấu chấm)	Mở repo trong trình soạn thảo web github.dev
`/`	Đưa con trỏ vào ô tìm kiếm
`?`	Hiện danh sách phím tắt của trang hiện tại

THEO DOCS "Keyboard shortcuts"[31].

BÀI TẬP · THAO TÁC FILE

1. Tạo thư mục `ghi-chu/` với file `ngay-1.md`.
2. Tải lên một ảnh nhỏ (dưới 1 MB) vào thư mục `anh/`.
3. Chèn ảnh đó vào README bằng `![mô tả](anh/ten-anh.png)`.
4. Sửa `ngay-1.md`, lần này chọn "tạo nhánh mới". Ta sẽ dùng nhánh này ở chương 9.

Kết quả cần đạt: Repo có thư mục mới, ảnh hiển thị trong README, và một nhánh mới.

07 CHƯƠNG 7

Commit và lịch sử thay đổi

Commit giúp bạn biết ai sửa gì, khi nào, vì sao, và quay lại bất kỳ lúc nào.

7.1 Commit gồm những gì?

- Các thay đổi trong file (thêm, xóa, sửa).
- Lời nhắn (commit message) mô tả thay đổi.
- Tác giả, thời gian, và một mã định danh duy nhất (SHA), ví dụ `a1b2c3d`.

THEO DOCS "About commits"[32].

7.2 Viết lời nhắn commit tốt

Nên	Không nên
Thêm mục Liên hệ vào README	update
Sửa lỗi chính tả trang giới thiệu	sua, abc
Đổi màu nút Đăng ký sang xanh	thay đổi nhiều thứ

- Dòng đầu ngắn (dưới khoảng 50–72 ký tự), viết như một mệnh lệnh: "Thêm…", "Sửa…", "Xóa…".
- Mỗi commit một ý. Sửa nhiều việc khác nhau thì tách thành nhiều commit.
- Cần giải thích thêm: để trống một dòng rồi viết phần mô tả chi tiết.

7.3 Xem lịch sử và so sánh thay đổi

1. Trên trang repo, bấm số **commits** (biểu tượng đồng hồ) phía trên danh sách file.
2. Bấm một commit để xem **diff**: dòng đỏ là bị xóa, dòng xanh là được thêm.
3. Trong một file, bấm **History** để xem riêng lịch sử file đó, hoặc **Blame** để xem từng dòng do commit nào tạo ra.

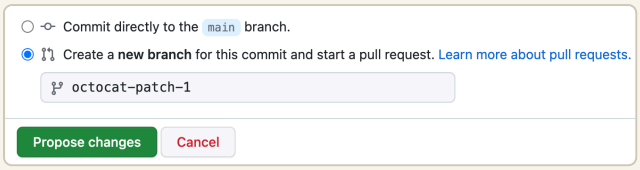


Diff của README.md: 3 dòng đỏ bị xóa, 3 dòng xanh được thêm.

Nguồn: ảnh chính thức GitHub Docs, trang "Hello World" [6]

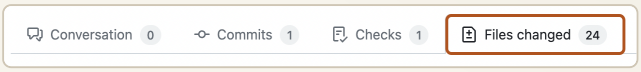
7.4 Commit trên nhánh nào, có tag nào?

Trang chi tiết commit cho biết commit nằm trên nhánh nào và có gắn tag (phiên bản) nào hay không[32].



Lựa chọn nhánh khi commit trên web.

Nguồn: ảnh chính thức GitHub Docs, trang "Creating new files" [28]



Tab "Files changed" trong pull request, nơi xem diff.

Nguồn: ảnh chính thức GitHub Docs, trang "Creating a pull request" [33]

7.5 Đồng tác giả (co-author)

Nếu hai người cùng làm một thay đổi, thêm dòng sau vào cuối lời nhắn commit (sau một dòng trống), GitHub sẽ ghi nhận cả hai[32]:

CUỐI COMMIT MESSAGE

```
Co-authored-by: Ten Ban <email-cua-ban@example.com>
```

7.6 Quay lại bản cũ

- Trên web: mở commit hoặc lịch sử file, bấm **Browse files** để xem toàn bộ repo tại thời điểm đó; sao chép nội dung cũ rồi commit lại.
- Với pull request đã merge: nút **Revert** trên PR tạo một PR mới đảo ngược thay đổi[34].
- Trên máy: `git revert <SHA>` tạo commit đảo ngược, an toàn vì không xóa lịch sử (chương 16).

NHỚ KỸ

Lịch sử Git là công khai với repo public. Lỡ commit mật khẩu hay khóa API thì xóa file ở commit sau là **chưa đủ**: khóa vẫn nằm trong lịch sử. Hãy thu hồi (revoke) khóa ngay, xem chương 25[35].

▶ VIDEO THAM KHẢO · TIẾNG ANH · QUÉT QR

Git Explained in 100 Seconds

xem từ 00:00 đến 01:57 kênh **Fireship** · 841.845 lượt xem (17/10/2026) · đăng 02/03/2020

Đoạn: toàn bộ video (6 chương ngắn). Ôn lại khái niệm commit (snapshot) và staging trong 2 phút.

youtu.be/hwP7WQkmECE?t=0

BÀI TẬP · ĐỌC LỊCH SỬ

1. Mở lịch sử commit của `hello-world`.
2. Chọn một commit, đọc diff.
3. Mở README, bấm **Blame**, xem dòng nào được sửa ở commit nào.
4. Viết một commit có dòng mô tả chi tiết ở dưới.

Kết quả cần đạt: Bạn đọc được diff và tìm được ai sửa dòng nào.

08 CHƯƠNG 8

Branch: làm việc song song

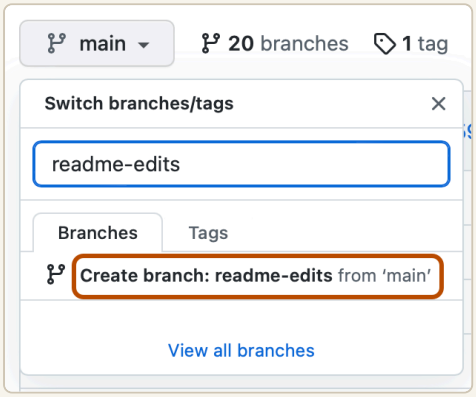
Nhánh cho phép thử nghiệm mà không ảnh hưởng bản chính. Làm hỏng? Xóa nhánh là xong.

8.1 Nhánh là gì?

Mỗi repo có một **nhánh mặc định** (thường là `main`). Khi tạo nhánh mới, bạn có một bản sao độc lập để sửa. Khi xong, mở pull request để gộp lại vào nhánh mặc định[36].

8.2 Tạo nhánh trên web

1. Trên trang repo, bấm menu nhánh (biểu tượng nhánh và chữ `main`).
2. Gõ tên nhánh mới, ví dụ `sua-readme`.
3. Bấm **Create branch: sua-readme from 'main'**.



Ô tạo nhánh mới "readme-edits" từ "main".
Nguồn: ảnh chính thức GitHub Docs, trang "Hello World" [6]

ĐẶT TÊN NHÁNH

- Mô tả mục đích: `them-trang-lien-he`, `sua-loi-dang-nhap`.
- Có thể thêm tiền tố: `feature/`, `fix/`, `docs/`.
- Chữ thường, không dấu, không khoảng trắng.

THEO DOCS Các bước theo "Hello World"[6].

8.3 Quản lý nhánh

- Xem tất cả nhánh: trang repo › bấm số **branches** cạnh menu nhánh.
- Xóa nhánh: biểu tượng thùng rác trong trang Branches; sau khi merge PR, GitHub cũng gợi ý nút **Delete branch**.
- Đổi nhánh mặc định: **Settings** › **General** › **Default branch** (cần quyền quản trị).

8.4 Bảo vệ nhánh chính

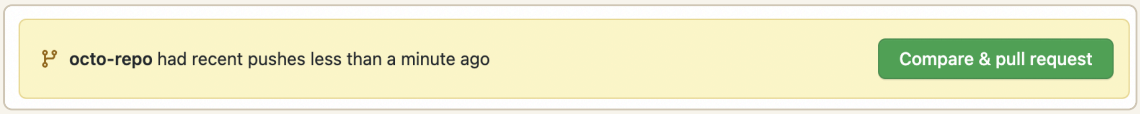
Khi làm nhóm, nên ngăn ai đó vô tình đẩy thẳng vào `main`. GitHub có hai công cụ[37][38]:

Công cụ	Làm được gì
Branch protection rules	Yêu cầu PR, số người duyệt tối thiểu, kiểm thử phải xanh, cấm force push, cấm xóa nhánh.
Rulesets	Bộ luật áp cho nhiều nhánh hoặc tag cùng lúc, bật tắt dễ, xem được ai được phép bỏ qua.

THEO DOCS Với repo **private**, nhánh được bảo vệ và yêu cầu người duyệt cần gói GitHub Pro, Team hoặc cao hơn; repo public dùng được với gói Free[21].

8.5 Chuyển nhánh và so sánh

- Chọn nhánh ở menu nhánh để xem file trên nhánh đó.
- So sánh hai nhánh: vào `github.com/<owner>/<repo>/compare`, chọn **base** và **compare**.
- Banner vàng "had recent pushes" hiện khi nhánh vừa có commit: bấm **Compare & pull request**.



Banner phía trên danh sách file, gợi ý mở pull request cho nhánh vừa push.

Nguồn: ảnh chính thức GitHub Docs, trang "Creating a pull request" [33]

▶ VIDEO THAM KHẢO · TIẾNG ANH · QUÉT QR

Git and GitHub Tutorial for Beginners

xem từ 23:12 đến 27:19 kênh **Kevin Stratvert** · 2.023.896 lượt xem (17/10/2026) · đăng 24/06/2022

Đoạn: "Branches". Tạo nhánh, chuyển nhánh và commit trên nhánh mới, làm mẫu từng bước.

youtu.be/tRZGeaHPoaw?t=1392

BÀI TẬP · NHÁNH ĐẦU TIÊN

1. Tạo nhánh `them-gioi-thieu` từ `main`.
2. Trên nhánh mới, sửa README thêm một đoạn giới thiệu.
3. Chuyển về `main`, kiểm tra README ở main chưa thay đổi.

Kết quả cần đạt: Bạn thấy rõ thay đổi chỉ nằm trên nhánh mới.

09 CHƯƠNG 9

Pull request

Pull request là trái tim của làm việc nhóm trên GitHub: mọi thay đổi được xem, góp ý và kiểm tra trước khi vào bản chính.

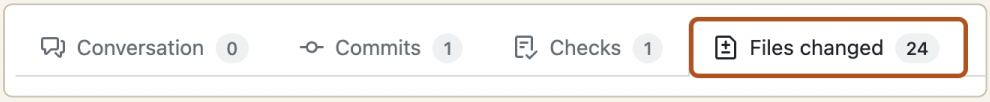
9.1 Pull request là gì?

Pull request (PR) là đề nghị gộp các commit từ một nhánh (**compare**, nhánh nguồn) vào nhánh khác (**base**, thường là `main`). PR hiển thị diff, nơi bình luận, kết quả kiểm thử tự động, và nút merge[39].

9.2 Tạo pull request

1. Vào tab **Pull requests** › **New pull request** (hoặc bấm **Compare & pull request** trên banner).
2. Chọn **base**: `main`, **compare**: nhánh của bạn.
3. Viết tiêu đề rõ ràng và mô tả: thay đổi gì, vì sao, cách kiểm tra.
4. Bên phải: chọn **Reviewers**, **Assignees**, **Labels** nếu cần.
5. Bấm **Create pull request**, hoặc mũi tên bên cạnh › **Create draft pull request** nếu chưa xong.

THEO DOCS "Creating a pull request"[33].



Các tab của pull request; tab "Files changed" chứa toàn bộ diff.
Nguồn: ảnh chính thức GitHub Docs, trang "Creating a pull request" [33]

9.3 Mô tả PR mẫu

MÔ TẢ PULL REQUEST

```
## Thay đổi
- Thêm mục "Liên hệ" vào README

## Lý do
Người xem chưa biết cách liên hệ nhóm.

## Cách kiểm tra
Mở README, kéo xuống cuối trang.

Closes #3
```

Dòng `Closes #3` sẽ tự đóng issue số 3 khi PR được merge vào nhánh mặc định[40].

9.4 Draft PR, thảo luận và cập nhật

- **Draft pull request**: đánh dấu "đang làm", không merge được cho tới khi bấm **Ready for review**.
- Muốn sửa theo góp ý: tiếp tục commit vào cùng nhánh, PR tự cập nhật.
- Tab **Conversation** chứa trao đổi; tab **Commits** liệt kê commit; tab **Checks** hiện kết quả Actions.

THEO DOCS "About pull requests"[39].

9.5 Stacked pull requests THAY ĐỔI 2026

Ngày 06/10/2026 GitHub công bố **stacked pull requests** chính thức (GA): chia một thay đổi lớn thành chuỗi PR nhỏ xếp chồng, review riêng từng PR và merge cùng nhau. Bản GA giữ nguyên phê duyệt khi "Rebase stack" nếu code không đổi[41]. Người mới chưa cần dùng; biết để không bỡ ngỡ khi gặp trong dự án lớn.

9.6 Copilot hỗ trợ PR

Copilot có thể tóm tắt PR, review code (chương 24), và khi repo bật Copilot cloud agent, nút **Fix with Copilot** trong khung merge có thể tự giải xung đột[42]. Các tính năng này phụ thuộc gói Copilot của bạn.

▶ VIDEO THAM KHẢO · TIẾNG ANH · QUÉT QR

How to create a pull request in 4 min | GitHub for Beginners

xem từ 00:59 đến 03:44 kênh **GitHub** · 331.685 lượt xem (17/10/2026) · đăng 12/08/2024

Đoạn: "Pull request demo" + "Tips". Video chính thức "GitHub for Beginners": demo tạo pull request và mẹo viết PR tốt.

youtu.be/nCKdihvneS0?t=59

BÀI TẬP · PR ĐẦU TIÊN

1. Dùng nhánh `them-gioi-thieu` ở chương 8 (hoặc nhánh tạo ở chương 6).
2. Mở PR vào `main` với mô tả theo mẫu 9.3.
3. Tự bình luận một dòng trong tab **Files changed**.
4. Chưa merge vội, chương 10 sẽ review và merge.

Kết quả cần đạt: Một PR đang mở, có mô tả đầy đủ.

10 CHƯƠNG 10

Code review và merge

Review là lúc nhóm cùng nhìn thay đổi trước khi vào bản chính. Biết review tốt giúp bạn được tin tưởng trong mọi dự án.

10.1 Ba kiểu review

Lựa chọn	Ý nghĩa
Comment	Góp ý chung, chưa duyệt cũng chưa yêu cầu sửa.
Approve	Đồng ý, thay đổi sẵn sàng để merge.
Request changes	Cần sửa trước khi merge.

THEO DOCS Ai có quyền đọc repo cũng review và bình luận được[43].

10.2 Review từng bước

1. Mở PR > tab **Files changed**.
2. Rê chuột vào một dòng, bấm dấu **+** màu xanh để bình luận đúng dòng đó.
3. Muốn đề xuất sửa cụ thể: bấm biểu tượng **Suggest changes** trong ô bình luận, sửa đoạn code; tác giả có thể bấm áp dụng ngay.
4. Đánh dấu file đã xem bằng ô **Viewed**.
5. Bấm **Review changes** (hoặc **Submit review**), chọn Comment / Approve / Request changes.

⚠ CHƯA CHẮC Tên nút trong khung review có thể khác chút giữa giao diện cũ và mới.

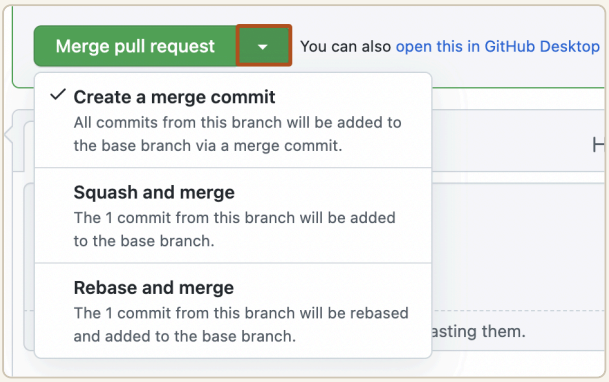
REVIEW LỊCH SỰ, HIỆU QUẢ

- Khen điểm tốt trước, góp ý sau.
- Góp ý vào code, không nhắm vào người.
- Đặt câu hỏi thay vì ra lệnh: “Mình nghĩ tách hàm này ra sẽ dễ đọc hơn, bạn thấy sao?”.
- Phân biệt “bắt buộc sửa” và “gợi ý nhỏ (nit)”.

10.3 Ba cách merge

Cách	Điều xảy ra	Khi nào dùng
Create a merge commit	Giữ mọi commit của nhánh và thêm một commit gộp (mặc định)	Muốn giữ đầy đủ lịch sử
Squash and merge	Gộp mọi commit của PR thành một commit	Lịch sử gọn, mỗi PR một commit
Rebase and merge	Đặt lại từng commit lên đầu nhánh base, không có commit gộp	Lịch sử thẳng, nhóm đã quen rebase

THEO DOCS "About merge methods on GitHub"[44]. Quản trị repo có thể chỉ cho phép một số cách.



Mũi tên bên cạnh nút merge mở danh sách các cách merge.
Nguồn: ảnh chính thức GitHub Docs, trang "Merging a pull request" [34]

10.4 Merge và dọn dẹp

1. Kiểm tra PR không còn xung đột và các kiểm thử (checks) đã xanh.
2. Bấm **Merge pull request** (hoặc cách đã chọn) › **Confirm merge**.
3. Bấm **Delete branch** để xóa nhánh đã xong.

THEO DOCS "Merging a pull request"[34].

10.5 Copilot code review

Bạn có thể thêm **Copilot** vào danh sách Reviewers để nhận nhận xét tự động. THAY ĐỔI 2026 Từ 01/06/2026 mỗi lần Copilot code review tốn AI credits và cả phút GitHub Actions (repo public vẫn miễn phí phút)[45][46]. Gói Copilot Free chỉ có "Review selection" trong VS Code[47].

BÀI TẬP · REVIEW VÀ MERGE

1. Nhờ bạn bè (hoặc tự bạn, bằng tài khoản thứ hai nếu có) review PR ở chương 9 với một gợi ý sửa.
2. Áp dụng gợi ý bằng nút **Commit suggestion**.
3. Merge bằng **Squash and merge**, rồi xóa nhánh.

Kết quả cần đạt: PR đã merge, nhánh đã xóa, main có thay đổi mới.

11 CHƯƠNG 11

Xử lý xung đột (merge conflict)

Xung đột không phải lỗi, chỉ là Git không tự quyết được giữ bản nào. Bạn chọn, Git làm phần còn lại.

11.1 Vì sao có xung đột?

Xung đột xảy ra khi hai nhánh cùng sửa **cùng một dòng** của một file, hoặc một bên sửa file mà bên kia xóa file đó. Git đánh dấu chỗ xung đột và chờ bạn quyết định[48].

DẤU HIỆU XUNG ĐỘT TRONG FILE

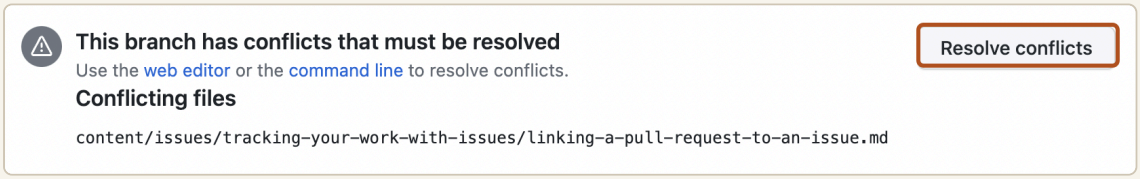
```
<<<<<<< main
Liên hệ: facebook.com/nhom-a
=======
Liên hệ: zalo nhóm lớp 12A
>>>>>>> them-lien-he
```

Phần trên `=======` là nội dung ở nhánh base, phần dưới là nhánh của bạn. Bạn giữ một bên, giữ cả hai hoặc viết lại, rồi **xóa cả ba dòng đánh dấu**.

11.2 Giải xung đột ngay trên GitHub

GitHub giải được trên web các xung đột đơn giản dạng "sửa cùng dòng". Loại khác cần dùng dòng lệnh hoặc GitHub Desktop[42].

1. Mở PR có cảnh báo xung đột, bấm **Resolve conflicts**.
2. Sửa nội dung, xóa các dấu `<<<<<<<`, `=======`, `>>>>>>>`.
3. Bấm **Mark as resolved** cho từng file.
4. Bấm **Commit merge**.



Cảnh báo pull request có xung đột và nút "Resolve conflicts".

Nguồn: ảnh chính thức GitHub Docs, trang "Resolving a merge conflict on GitHub" [42]



Trình sửa xung đột trên GitHub, nút "Mark as resolved".

Nguồn: ảnh chính thức GitHub Docs, trang "Resolving a merge conflict on GitHub"[42]

LƯU Ý TỪ DOCS

Giải xung đột trên GitHub sẽ merge **toàn bộ** nhánh base vào nhánh của bạn. Nếu nhánh của bạn là nhánh mặc định hoặc được bảo vệ, GitHub có thể yêu cầu tạo nhánh mới[42].

11.3 Giải xung đột trên máy

DÒNG LỆNH

```
git checkout them-lien-he
git merge main            # Git báo CONFLICT
# mở file, sửa, xóa dấu đánh dấu, lưu lại
git add README.md
git commit -m "Giải xung đột phần Liên hệ"
git push
```

THEO DOCS "Resolving a merge conflict using the command line"[49]. Trong VS Code và GitHub Desktop có nút chọn "Accept Current / Incoming / Both".

11.4 Phòng xung đột

- PR nhỏ, merge sớm; đừng để nhánh sống quá lâu.
- Cập nhật nhánh của bạn từ `main` thường xuyên (nút **Update branch** trên PR).
- Chia việc rõ: mỗi người một file hoặc một phần.

▶ VIDEO THAM KHẢO · TIẾNG ANH · QUÉT QR

Git & GitHub Tutorial for Beginners #9 - Merging Branches (& conflicts)

xem từ 03:38 đến 08:20 kênh **Net Ninja** · 501.915 lượt xem (17/10/2026) · đăng 15/06/2017

Đoạn: "Simulating a merge conflict" + "Resolving merge conflicts". Mô phỏng một xung đột và cách giải từng bước trên dòng lệnh.

youtu.be/XX-Kct0PfFc?t=218

BÀI TẬP · TẠO VÀ GIẢI XUNG ĐỘT

1. Tạo hai nhánh `a` và `b` từ `main`.
2. Trên mỗi nhánh, sửa **cùng một dòng** trong README theo hai cách khác nhau.
3. Mở PR `a`, merge. Mở PR `b`: GitHub báo xung đột.
4. Bấm **Resolve conflicts**, giữ cả hai ý, commit và merge.

Kết quả cần đạt: Bạn đã tự gây và tự giải một xung đột.

12 CHƯƠNG 12

Issues, labels và milestones

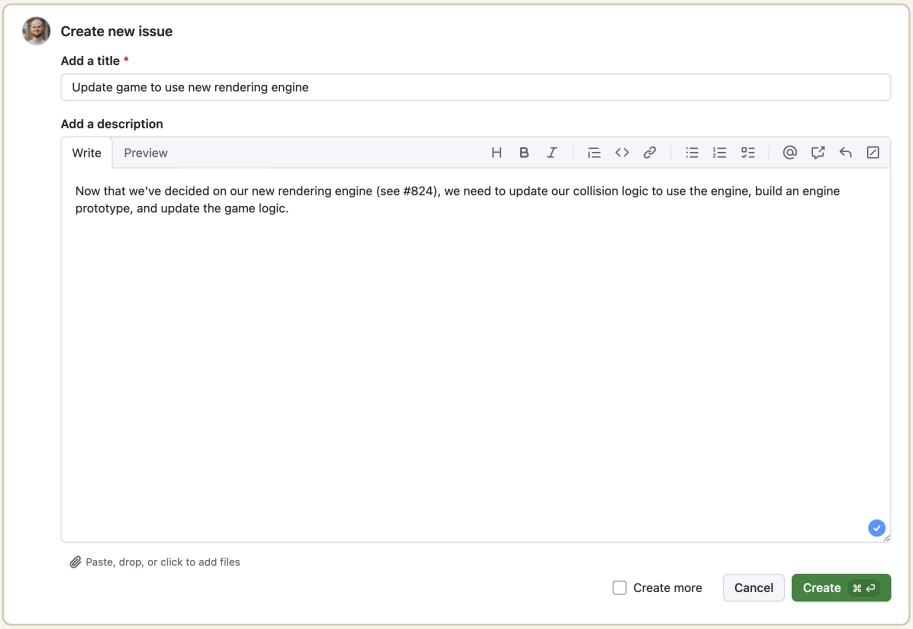
Issue là "phiếu việc" của GitHub. Dùng tốt Issues, bạn quản lý được cả công việc không liên quan đến code.

12.1 Issue dùng để làm gì?

Issue ghi lại lỗi, tính năng mới, ý tưởng hay bất cứ việc gì cần thảo luận. Bạn có thể chia việc lớn thành **sub-issues** (issue con), đặt quan hệ chặn nhau (**issue dependencies**), gắn loại issue, nhãn và mốc[40].

12.2 Tạo issue

1. Vào tab **Issues** › **New issue** (repo có mẫu sẽ cho chọn mẫu).
2. Viết tiêu đề ngắn, rõ; phần mô tả dùng Markdown, có thể thêm danh sách việc.
3. Bên phải: **Assignees** (người phụ trách), **Labels**, **Type**, **Projects**, **Milestone**.
4. Bấm **Create**.



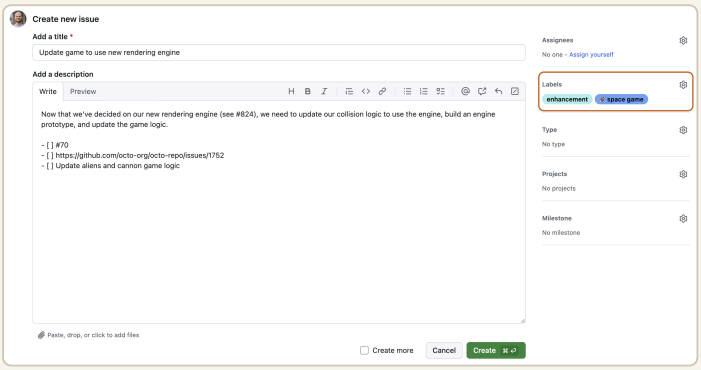
Biểu mẫu issue mới với tiêu đề và phần mô tả.
Nguồn: ảnh chính thức GitHub Docs, trang "Quickstart for GitHub Issues" [50]

THEO DOCS "Quickstart for GitHub Issues"[50]. Ô **Type** chỉ hiện khi tổ chức đã cấu hình loại issue.

12.3 Nhãn (labels) có sẵn

Nhãn	Ý nghĩa
`bug`	Lỗi, hành vi không mong muốn
`documentation`	Cần bổ sung, cải thiện tài liệu
`enhancement`	Đề xuất tính năng mới
`good first issue`	Việc phù hợp người mới đóng góp lần đầu
`help wanted`	Người duy trì cần giúp đỡ
`question`	Cần thêm thông tin
`duplicate`, `invalid`, `wontfix`	Trùng lặp, không còn liên quan, sẽ không làm
`accessibility`	Rào cản với người khuyết tật

THEO DOCS Danh sách nhãn mặc định trong "Managing labels"[51]. Tạo nhãn riêng: tab **Issues** › **Labels** › **New label**.



Mục "Labels" ở thanh bên phải của biểu mẫu issue.

Nguồn: ảnh chính thức GitHub Docs, trang "Quickstart for GitHub Issues" [50]

12.4 Milestones: gom việc theo mốc

Milestone gom issue và PR theo một mục tiêu có hạn (ví dụ "Ra mắt trang web 30/11"). GitHub hiển thị thanh tiến độ theo số việc đã đóng[52].

12.5 Liên kết issue với PR

- Viết `Closes #12`, `Fixes #12` hoặc `Resolves #12` trong mô tả PR: merge vào nhánh mặc định sẽ tự đóng issue 12.
- Nhắc `#12` ở bất kỳ đâu sẽ tạo liên kết qua lại.
- Nhắc `@username` để gọi người vào thảo luận (họ nhận thông báo).

BÀI TẬP · BẢNG VIỆC NHÓM

1. Tạo 3 issue: "Viết trang giới thiệu", "Thêm ảnh đại diện", "Sửa lỗi chính tả".
2. Gắn nhãn phù hợp; tạo nhãn mới `ưu tiên` màu đỏ.
3. Tạo milestone "Phiên bản 1" và gán 3 issue vào.
4. Mở một PR có `Closes #<số>`, merge, xem issue tự đóng.

Kết quả cần đạt: Issue tự đóng khi PR merge; milestone hiển thị tiến độ.

13 CHƯƠNG 13

GitHub Projects

Projects biến issue và PR thành bảng kế hoạch trực quan, cập nhật tự động hai chiều.

13.1 Projects là gì?

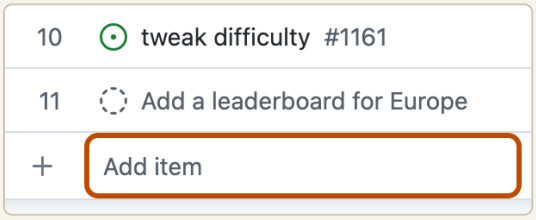
Một project là bảng tính, bảng kanban và lộ trình (roadmap) gắn trực tiếp với issue và PR, ở cấp tài khoản cá nhân hoặc tổ chức. Đổi người phụ trách trong project thì issue cũng đổi theo, và ngược lại[53].

Bố cục (layout)	Phù hợp
Table	Danh sách dày thông tin, lọc, sắp xếp, nhóm
Board	Kanban: kéo thẻ qua các cột Todo, In Progress, Done
Roadmap	Dòng thời gian theo ngày bắt đầu, kết thúc hoặc iteration

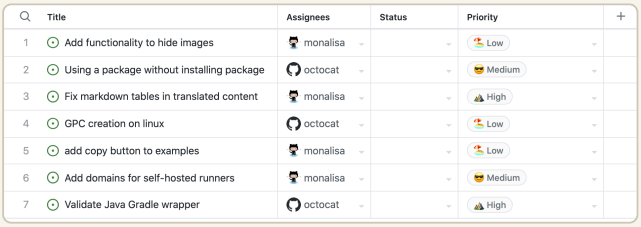
13.2 Tạo project đầu tiên

1. Vào hồ sơ của bạn › tab **Projects** › **New project**.
2. Chọn mẫu (ví dụ Board) hoặc bắt đầu trống, đặt tên.
3. Ở dòng cuối bảng, dùng ô **Add item**: gõ `#` để chọn issue/PR, hoặc gõ chữ để tạo **draft issue**.
4. Thêm trường tùy chỉnh bằng **+** (Add field): ngày, số, lựa chọn, iteration.
5. Tạo thêm view (**New view**) cho từng góc nhìn.

THEO DOCS "Quickstart for Projects"[54].



Ô "Add item" ở dòng cuối của view dạng bảng.
Nguồn: ảnh chính thức GitHub Docs, trang "Quickstart for Projects" [54]



Danh sách issue có trường ưu tiên Low, Medium, High.
Nguồn: ảnh chính thức GitHub Docs, trang "Quickstart for Projects" [54]

13.3 Tự động hóa và biểu đồ

- **Workflows có sẵn**: ví dụ tự đặt Status = Done khi issue đóng hoặc PR merge. Vào menu ☰ của project › **Workflows**.
- **Insights**: biểu đồ số việc theo trạng thái, theo người, theo thời gian.
- **Status updates**: ghi nhanh tình hình dự án cho cả nhóm.

THEO DOCS "About Projects"[53].

13.4 Gợi ý dùng Projects khi không code

Tình huống	Cách dựng
Kế hoạch nội dung kênh YouTube	Board: Ý tưởng › Đang quay › Dựng › Đã đăng; trường "Ngày đăng"
Đồ án nhóm ở trường	Table nhóm theo người phụ trách, milestone theo tuần
Lộ trình học GitHub	Roadmap theo tuần, mỗi chương sách là một draft issue

▶ VIDEO THAM KHẢO · TIẾNG ANH · QUÉT QR

How to use GitHub issues and projects | GitHub for Beginners

xem từ 01:15 đến 05:20 kênh **GitHub** · 69.263 lượt xem (17/10/2026) · đăng 02/03/2026

Đoạn: "What are GitHub issues?" đến hết "Managing your project board". Video chính thức: issue là gì, project là gì và cách quản lý bảng project.

youtu.be/c67GaAkf1BE?t=75

BÀI TẬP · BẢNG KANBAN

1. Tạo project dạng Board tên "Học GitHub".
2. Thêm 3 issue ở chương 12 và 3 draft issue mới.
3. Thêm trường "Hạn chót" kiểu Date.
4. Bật workflow "Item closed › Done"; đóng một issue và xem thẻ tự sang cột Done.

Kết quả cần đạt: Bảng kanban tự cập nhật khi issue đóng.

14 CHƯƠNG 14

GitHub Discussions

Không phải câu hỏi nào cũng là "việc cần làm". Discussions là nơi hỏi đáp, chia sẻ, bình chọn mà không làm rối danh sách issue.

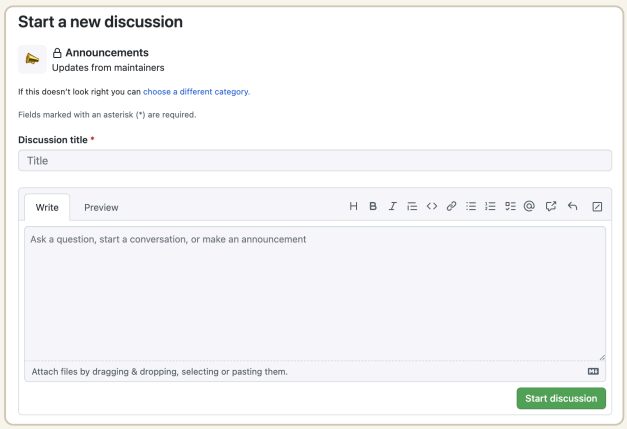
14.1 Issues hay Discussions?

	Issues	Discussions
Mục đích	Việc cụ thể cần làm, theo dõi được	Trao đổi mở: hỏi đáp, ý tưởng, thông báo
Kết thúc	Đóng khi xong	Đánh dấu câu trả lời đúng (Q&A), hoặc để mở
Ví dụ	"Nút Đăng ký bị lỗi trên điện thoại"	"Mọi người dùng theme nào cho trang web?"

THEO DOCS Discussions dành cho trao đổi cần minh bạch nhưng không cần theo dõi như công việc[55].

14.2 Bật Discussions và tạo chủ đề

1. Repo › **Settings** › **General** › mục **Features** › tích **Discussions** (cần quyền ghi trở lên; dùng được cho repo public và private).
2. Vào tab **Discussions** › **New discussion**, chọn thể loại (Announcements, Q&A, Ideas, Polls…).
3. Viết tiêu đề và nội dung, bấm **Start discussion**.



Ô tiêu đề và nội dung khi tạo discussion mới.

Nguồn: ảnh chính thức GitHub Docs, trang "Quickstart for GitHub Discussions" [55]

THEO DOCS Có thể chuyển một issue thành discussion và ngược lại; tổ chức có thể bật Discussions cấp tổ chức[56].

BÀI TẬP · HỎI ĐÁP

1. Bật Discussions cho `hello-world`.
2. Tạo một discussion Q&A và một Poll.
3. Trả lời câu hỏi của chính bạn và bấm **Mark as answer**.

Kết quả cần đạt: Tab Discussions có câu hỏi đã được đánh dấu trả lời.

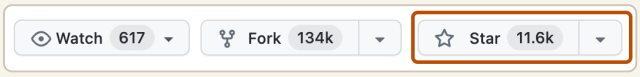
15 CHƯƠNG 15

Fork, Star, Watch và đóng góp mã nguồn mở

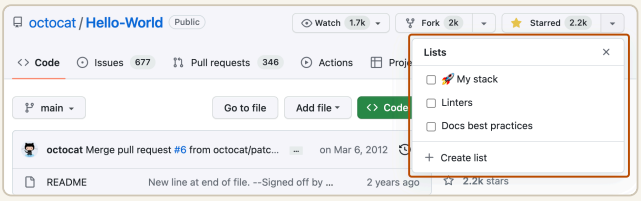
Ba nút ở góc phải trang repo: Watch để nhận tin, Fork để sao chép và đóng góp, Star để lưu và cảm ơn tác giả.

15.1 Star: lưu và cảm ơn

Bấm **Star** để lưu repo vào trang sao của bạn và bày tỏ cảm ơn tác giả. Nhiều bảng xếp hạng và trang Explore dựa vào số sao. Bạn có thể xếp repo đã gắn sao vào các **danh sách (lists)**[57].



Nút "Star" trên thanh repo.
Nguồn: ảnh chính thức GitHub Docs, trang "Saving repositories with stars" [57]



Menu Star mở rộng: thêm repo vào danh sách.
Nguồn: ảnh chính thức GitHub Docs, trang "Saving repositories with stars" [57]

15.2 Watch: chọn mức thông báo

Mức	Bạn nhận thông báo khi
Participating and @mentions	Chỉ khi bạn tham gia hoặc được nhắc tên (mặc định với repo không phải của bạn)
All Activity	Mọi hoạt động trong repo
Ignore	Không nhận gì, kể cả khi được nhắc tên
Custom	Chọn loại sự kiện: issue, PR, release, discussion, cảnh báo bảo mật…

THEO DOCS "About notifications"[58] và "Viewing your subscriptions"[59]. ⚠ CHƯA CHẮC Tên các mức hiển thị có thể đổi nhẹ.

15.3 Follow người khác

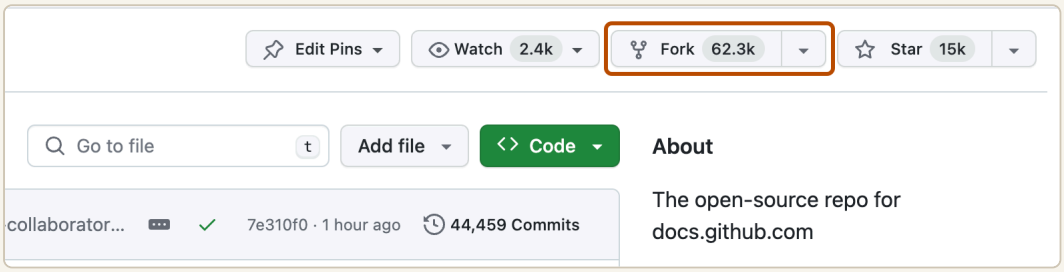
Bấm **Follow** trên hồ sơ một người để thấy hoạt động công khai của họ trên bảng tin (Feed)[60].

15.4 Fork khác branch thế nào?

Branch là một phần của một repo. Fork là **một repo riêng** sao chép từ repo gốc (upstream), có cài đặt và quyền riêng nhưng vẫn kết nối với repo gốc để gửi PR ngược lại[61]. Bạn fork khi muốn đóng góp vào repo mà mình không có quyền ghi.

15.5 Quy trình đóng góp đầu tiên

1. Vào repo gốc, bấm **Fork** › **Create fork**.
2. Trong bản fork của bạn: tạo nhánh, sửa, commit (trên web hoặc clone về máy).
3. Bấm **Contribute** › **Open pull request**: PR sẽ đi từ fork của bạn về repo gốc.
4. Theo dõi góp ý của người duy trì, sửa tiếp trên cùng nhánh.
5. Giữ fork cập nhật: nút **Sync fork** trên trang fork ⚠ CHƯA CHẮC .



Nút "Fork" trên trang chính của repo.
Nguồn: ảnh chính thức GitHub Docs, trang "Contributing to a project" [62]

THEO DOCS "Contributing to a project"[62].

TÌM VIỆC CHO NGƯỜI MỚI

- Lọc issue theo nhãn `good first issue` hoặc `help wanted`.
- Đọc `CONTRIBUTING.md` và `CODE_OF_CONDUCT.md` của dự án trước khi làm.
- Hỏi trong issue "Mình nhận việc này được không?" trước khi bắt tay vào làm.
- Sửa lỗi chính tả trong tài liệu là đóng góp đầu tiên rất phổ biến.

BÀI TẬP · ĐÓNG GÓP THỬ

1. Fork repo `octocat/Spoon-Knife` (repo GitHub dùng để thực hành fork).
2. Tạo một file mới trong fork, commit.
3. Mở PR về repo gốc rồi tự đóng PR (đây chỉ là bài tập).
4. Gắn sao 3 repo bạn thấy hữu ích, tạo list "Học tập".

Kết quả cần đạt: Bạn biết trọn quy trình fork, sửa, mở PR.

⚠ CHƯA CHẮC Repo `octocat/Spoon-Knife` được docs dùng làm ví dụ fork; PR thật gửi vào đó có thể không được xử lý.

16 CHƯƠNG 16

Git trên máy tính

Khi đã quen web, đưa repo về máy giúp bạn làm việc offline, dùng trình soạn thảo yêu thích và thao tác nhanh hơn.

16.1 Cài và cấu hình Git

1. Tải và cài bản Git mới nhất tại `git-scm.com/downloads` (Windows, macOS, Linux).
2. Mở Terminal (macOS/Linux) hoặc Git Bash / PowerShell (Windows), kiểm tra: `git --version`.
3. Đặt tên và email gắn với commit (chỉ làm một lần):

CẤU HÌNH LẦN ĐẦU

```
git config --global user.name "Ten Cua Ban"
git config --global user.email "<email-cua-ban>"
git config --global user.name     # xem lại tên đã đặt
```

THEO DOCS "Set up Git"[63] và "Setting your username in Git"[64]. Muốn giấu email thật, GitHub cung cấp địa chỉ `noreply` trong **Settings** › **Emails**.

16.2 Đăng nhập GitHub từ Git

GitHub **không nhận mật khẩu tài khoản** khi push qua HTTPS. Có ba cách, docs khuyên dùng GitHub CLI hoặc Git Credential Manager cho HTTPS[65]:

Cách	Làm thế nào	Hợp với
GitHub CLI	`gh auth login`, chọn HTTPS, trả lời Y khi hỏi dùng thông tin GitHub cho Git	Người mới (dễ nhất)
Git Credential Manager	Đi kèm Git cho Windows; lần đầu push sẽ mở trình duyệt để đăng nhập	Người dùng Windows
SSH key	Tạo khóa `ssh-keygen -t ed25519 -C "<email>"`, thêm khóa công khai vào Settings › SSH and GPG keys	Người dùng lâu dài

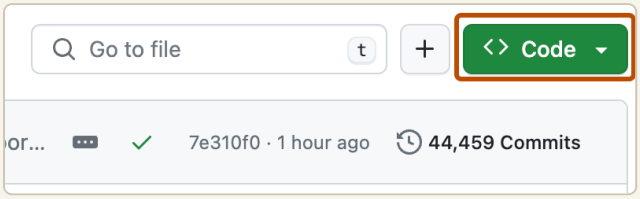
THEO DOCS SSH: "About SSH"[66], "Generating a new SSH key"[67]. Token (PAT) chỉ dùng khi thật cần, ưu tiên loại **fine-grained**[68].

TOKEN LÀ MẬT KHẨU

Personal access token (dạng `<YOUR_TOKEN>`) có quyền như mật khẩu. Đặt hạn dùng, chỉ cấp quyền tối thiểu, không dán vào chat, file, ảnh chụp màn hình. Lộ token: vào **Settings** › **Developer settings** › **Personal access tokens** và xóa ngay.

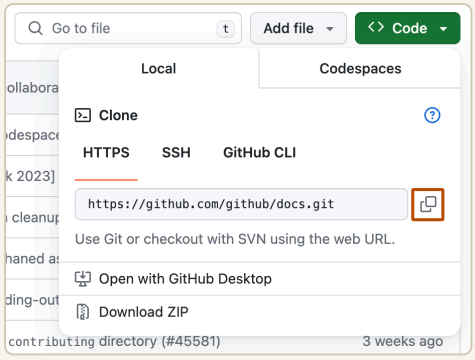
16.3 Clone: tải repo về máy

1. Trên trang repo, bấm nút xanh **Code**.
2. Chọn tab HTTPS (hoặc SSH, GitHub CLI), bấm biểu tượng sao chép.
3. Trong Terminal, chuyển tới thư mục muốn lưu, chạy `git clone <URL>`.



Nút "Code" trên trang repo.

Nguồn: ảnh chính thức GitHub Docs, trang "Cloning a repository" [69]



Menu "Code": sao chép URL HTTPS.

Nguồn: ảnh chính thức GitHub Docs, trang "Cloning a repository" [69]

THEO DOCS "Cloning a repository"[69].

16.4 Vòng làm việc hằng ngày

TỪ PULL TỚI PUSH

```
git pull                        # 1. lấy thay đổi mới nhất về
git switch -c them-trang-moi    # 2. tạo và chuyển sang nhánh mới
# 3. sửa file bằng trình soạn thảo
git status                      # 4. xem file nào đã đổi
git add .                       # 5. đưa thay đổi vào vùng chờ (staging)
git commit -m "Thêm trang mới" # 6. tạo commit
git push -u origin them-trang-moi  # 7. đẩy nhánh lên GitHub
```

Sau bước 7, vào GitHub sẽ thấy banner gợi ý mở pull request. Docs: "Pushing commits to a remote repository"[70], "Getting changes from a remote repository"[71].

16.5 Đưa thư mục có sẵn lên GitHub

REPO RỖNG TRÊN GITHUB, CODE CÓ SẴN TRÊN MÁY

```
cd thu-muc-du-an
git init -b main
git add .
git commit -m "Commit đầu tiên"
git remote add origin https://github.com/<username>/<ten-repo>.git
git push -u origin main
```

Cách nhanh hơn: `gh repo create` và làm theo hướng dẫn trên màn hình (chương 18)[72].

▶ VIDEO THAM KHẢO · TIẾNG ANH · QUÉT QR

Git and GitHub Tutorial for Beginners

xem từ 35:09 đến 37:29 kênh **Kevin Stratvert** · 2.023.896 lượt xem (17/10/2026) · đăng 24/06/2022

Đoạn: "Push local repo to GitHub". Kevin Stratvert: đẩy repo có sẵn trên máy lên GitHub.

youtu.be/tRZGeaHPoaw?t=2109

16.6 Lệnh Git hay dùng

Lệnh	Tác dụng
`git status`	Xem trạng thái: file đổi, file chưa theo dõi, nhánh hiện tại
`git log --oneline`	Lịch sử commit gọn, mỗi commit một dòng
`git diff`	Xem chi tiết thay đổi chưa stage
`git branch`	Liệt kê nhánh; `git branch -d ten` để xóa nhánh đã merge
`git switch ten-nhanh`	Chuyển nhánh
`git merge ten-nhanh`	Gộp nhánh khác vào nhánh hiện tại
`git fetch`	Tải thông tin mới từ GitHub nhưng chưa gộp
`git restore ten-file`	Bỏ thay đổi chưa commit của một file
`git revert <SHA>`	Tạo commit đảo ngược một commit cũ (an toàn)
`git remote -v`	Xem repo GitHub mà thư mục đang kết nối

THEO DOCS Tổng quan Git: "About Git"[2].

BA VÙNG CỦA GIT

Working directory (thư mục bạn đang sửa) › `git add` › **Staging area** (vùng chờ) › `git commit` › **Repository** (lịch sử) › `git push` › **GitHub**.

▶ VIDEO THAM KHẢO · TIẾNG ANH · QUÉT QR

A brief introduction to Git for beginners | GitHub

xem từ 05:10 đến 08:29 kênh **GitHub** · 2.225.973 lượt xem (17/10/2026) · đăng 26/05/2024

Đoạn: "Configuring Git" + "Basic Terminal and Git Commands". Video chính thức: cấu hình Git và các lệnh terminal cơ bản.

youtu.be/r8jQ9hVA2qs?t=310

▶ VIDEO THAM KHẢO · TIẾNG ANH · QUÉT QR

Git and GitHub for Beginners - Crash Course

xem từ 14:30 đến 17:30 kênh **freeCodeCamp.org** · 5.160.077 lượt xem (17/10/2026) · đăng 28/05/2020

Đoạn: "cloning through VS Code". freeCodeCamp: clone repo bằng VS Code, bước đầu làm việc trên máy.

youtu.be/RGOj5yH7evk?t=870

BÀI TẬP · TỪ MÁY LÊN GITHUB

1. Cài Git, cấu hình tên và email.
2. Đăng nhập bằng `gh auth login` (cài gh ở chương 18) hoặc Git Credential Manager.
3. Clone `hello-world`, sửa README, commit và push lên một nhánh mới.
4. Mở PR trên web, merge, rồi `git switch main` và `git pull`.

Kết quả cần đạt: Thay đổi đi trọn vòng: máy › GitHub › máy.

17 CHƯƠNG 17

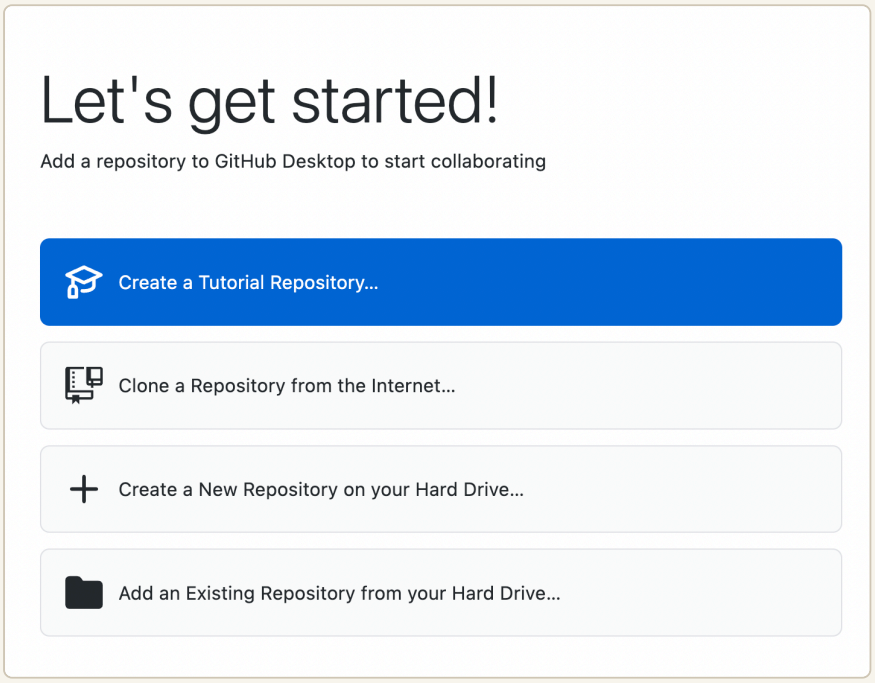
GitHub Desktop

GitHub Desktop là ứng dụng miễn phí của GitHub: commit, tạo nhánh, push, pull bằng nút bấm.

17.1 Cài đặt và đăng nhập

1. Tải tại `desktop.github.com`. Hỗ trợ **macOS 12.0 trở lên** và **Windows 10 64-bit trở lên**.
2. Mở ứng dụng, đăng nhập: macOS vào **GitHub Desktop** › **Settings** › **Accounts**; Windows vào **File** › **Options** › **Accounts** › **Sign In**.
3. Trình duyệt mở ra để xác nhận; quay lại ứng dụng.

THEO DOCS "Installing GitHub Desktop"[73] và "Getting started with GitHub Desktop"[74]. Docs không liệt kê bản Linux chính thức.

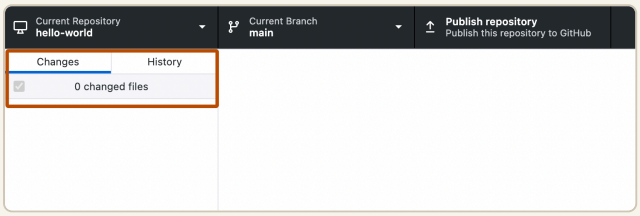


Màn hình "Let's get started!" của GitHub Desktop.
Nguồn: ảnh chính thức GitHub Docs, trang "Creating your first repository using GitHub Desktop"[75]

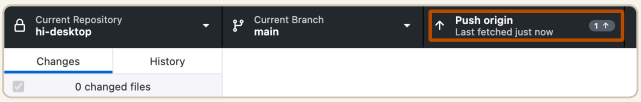
17.2 Vòng làm việc trong GitHub Desktop

1. **Lấy repo**: **File** › **Clone repository**, hoặc trên web bấm **Code** › **Open with GitHub Desktop**. Tạo mới: **File** › **New repository**.
2. **Sửa file**: **Repository** › **Open in Visual Studio Code** (hoặc trình soạn thảo bạn chọn).
3. **Xem thay đổi**: tab **Changes** bên trái, tích chọn file muốn commit.
4. **Commit**: gõ lời nhắn ở ô dưới cùng bên trái, bấm **Commit to main** (hoặc tên nhánh).
5. **Push**: bấm **Push origin** trên thanh trên cùng. Repo mới tạo thì bấm **Publish repository**.
6. **Nhánh và PR**: menu **Current branch** › **New branch**; sau khi push, bấm **Create Pull Request**.

THEO DOCS "Creating your first repository using GitHub Desktop"[75].



Thanh bên trái với hai tab "Changes" và "History".
Nguồn: ảnh chính thức GitHub Docs, trang "Creating your first repository using GitHub Desktop" [75]



Nút "Push origin" trên thanh repo.
Nguồn: ảnh chính thức GitHub Docs, trang "Creating your first repository using GitHub Desktop" [75]

KHI NÀO DÙNG DESKTOP?

- Bạn chưa quen dòng lệnh; muốn nhìn rõ diff trước khi commit.
- Giải xung đột bằng nút bấm, mở file trong trình soạn thảo.
- Tab History để xem lại, hoàn tác (revert) commit.

▶ VIDEO THAM KHẢO · TIẾNG ANH · QUÉT QR

Git, GitHub, & GitHub Desktop for beginners

xem từ 02:48 đến 04:12 kênh **Coder Coder** · 1.038.533 lượt xem (17/10/2026) · đăng 01/02/2022
Đoạn: "Set up GitHub Desktop". Coder Coder: cài đặt và thiết lập GitHub Desktop cho người mới.
youtu.be/8Dd7KRpKeaE?t=168

BÀI TẬP · DESKTOP

1. Cài GitHub Desktop, đăng nhập.
2. Clone `hello-world` qua nút "Open with GitHub Desktop".
3. Sửa README, commit, Push origin.
4. Tạo nhánh mới, sửa, mở Pull Request từ Desktop.

Kết quả cần đạt: Một PR được tạo hoàn toàn từ GitHub Desktop.

18 CHƯƠNG 18

GitHub CLI (gh)

gh đưa pull request, issue, repo, Codespaces vào Terminal. Rất tiện khi bạn đã dùng Git trên máy.

18.1 Cài đặt và đăng nhập

Hệ điều hành	Lệnh cài (theo repo cli/cli)
Windows	`winget install --id GitHub.cli --source winget`
macOS	`brew install gh`
Linux, cách khác	Xem hướng dẫn tại `cli.github.com`

Nguồn lệnh cài: [76]. Tổng quan gh: "About GitHub CLI"[77].

ĐĂNG NHẬP

```
gh auth login      # làm theo hướng dẫn, chọn GitHub.com, HTTPS, đăng nhập qua trình duyệt
gh auth status     # kiểm tra đã đăng nhập chưa
```

18.2 Lệnh gh thường dùng

Lệnh	Tác dụng
`gh status`	Tổng hợp việc của bạn trên các repo đang theo dõi
`gh repo create`	Tạo repo mới (có thể đẩy thư mục hiện tại lên)
`gh repo clone OWNER/REPO`	Clone repo
`gh repo view OWNER/REPO --web`	Mở repo trong trình duyệt
`gh issue list` / `gh issue create`	Xem / tạo issue
`gh issue list --assignee "@me"`	Issue được giao cho bạn
`gh pr create`	Tạo pull request từ nhánh hiện tại
`gh pr list --author "@me"`	PR do bạn tạo
`gh search prs --review-requested=@me --state=open`	PR đang chờ bạn review
`gh codespace create` / `list`	Tạo / liệt kê codespace
`gh <lệnh> --help`	Trợ giúp cho từng lệnh

THEO DOCS Các lệnh theo "GitHub CLI quickstart"[72].

18.3 Tùy chỉnh gh

CẤU HÌNH

```
gh config set editor "code -w"      # dùng VS Code để soạn nội dung issue, PR
gh alias set prd "pr create --draft" # gõ gh prd để mở PR nháp
gh auth switch                      # đổi giữa nhiều tài khoản
```

THEO DOCS "GitHub CLI quickstart"[72].

18.4 gh và Copilot CLI khác nhau

gh là công cụ dòng lệnh để thao tác với GitHub (repo, issue, PR). **GitHub Copilot CLI** là trợ lý AI chạy trong Terminal, cài riêng (chương 24). Hai công cụ dùng song song được.

18.5 Một vòng PR bằng gh

TỪ CLONE TỚI MERGE

```
gh repo clone <username>/hello-world
cd hello-world
git switch -c sua-tieu-de
# sửa README.md
git commit -am "Sửa tiêu đề README"
git push -u origin sua-tieu-de
gh pr create --fill        # lấy tiêu đề, mô tả từ commit
gh pr view --web           # mở PR trên trình duyệt
gh pr merge                # chọn cách merge
```

⚠ CHƯA CHẮC Các tùy chọn `--fill`, `pr view --web`, `pr merge` có trong sổ tay gh tại `cli.github.com/manual`[78]; chạy `gh pr create --help` để xem chính xác trên bản bạn cài.

BÀI TẬP · CLI

1. Cài gh, chạy `gh auth login`.
2. Chạy `gh status` và `gh issue list` trong thư mục repo.
3. Tạo issue bằng `gh issue create`.
4. Làm trọn vòng PR ở 18.5.

Kết quả cần đạt: Một PR được tạo và merge từ Terminal.

19 CHƯƠNG 19

.gitignore và file không nên đưa lên

Không phải file nào cũng nên lên GitHub: file tạm, thư viện tải về, và đặc biệt là file chứa mật khẩu, khóa.

19.1 .gitignore là gì?

`.gitignore` là file văn bản ở gốc repo, liệt kê những file, thư mục Git sẽ bỏ qua. GitHub duy trì bộ mẫu chính thức trong repo `github/gitignore`; khi tạo repo trên web bạn có thể chọn mẫu ngay[79].

.GITIGNORE MẪU

```
# Hệ điều hành
.DS_Store
Thumbs.db

# Thư viện, bản build
node_modules/
dist/

# Bí mật: KHÔNG BAO GIỜ đưa lên
.env
*.pem
secrets.json
```

Mẫu	Ý nghĩa
`ten-file.txt`	Bỏ qua file có tên này ở mọi thư mục
`thu-muc/`	Bỏ qua cả thư mục
`*.log`	Bỏ qua mọi file đuôi .log
`!quan-trong.log`	Ngoại lệ: vẫn theo dõi file này
`# ...`	Dòng chú thích

19.2 Lỡ commit rồi thì sao?

Thêm tên file vào `.gitignore` không làm Git ngừng theo dõi file đã commit trước đó. Chạy lệnh sau để bỏ theo dõi (file vẫn còn trên máy)[79]:

BỎ THEO DÕI FILE ĐÃ COMMIT

```
git rm --cached TEN-FILE
git commit -m "Ngừng theo dõi TEN-FILE"
```

NẾU FILE ĐÓ CHỨA BÍ MẬT

Xóa khỏi lần commit sau là chưa đủ: bí mật vẫn nằm trong lịch sử. Việc đầu tiên là **thu hồi và tạo khóa mới** ở dịch vụ đã cấp khóa; sau đó mới tính tới việc xóa khỏi lịch sử theo hướng dẫn "Removing sensitive data from a repository"[35].

19.3 .gitignore toàn cục

Muốn mọi repo trên máy đều bỏ qua một số file (ví dụ `.DS_Store`), ghi chúng vào file `~/.config/git/ignore`[79].

19.4 Danh sách nên bỏ qua

Nhóm	Ví dụ
Bí mật	`.env`, khóa `*.pem`, file cấu hình chứa mật khẩu
Phụ thuộc tải về	`node_modules/`, `venv/`, `vendor/`
Bản build	`dist/`, `build/`, `*.exe`
File hệ thống, trình soạn thảo	`.DS_Store`, `Thumbs.db`, `.idea/`
Dữ liệu lớn, cá nhân	Video gốc, bảng tính khách hàng, ảnh CMND



▶ VIDEO THAM KHẢO · TIẾNG ANH · QUÉT QR

Git and GitHub Tutorial for Beginners

xem từ 09:18 đến 11:23 kênh **Kevin Stratvert** · 2.023.896 lượt xem (17/10/2026) · đăng 24/06/2022

Đoạn: "Ignore files with .gitignore". Kevin Stratvert: tạo .gitignore và kiểm tra file đã bị bỏ qua.

youtu.be/tRZGeaHPoaw?t=558

BÀI TẬP · DỌN REPO

1. Tạo file `.gitignore` trong `hello-world` với mẫu trên.
2. Tạo file `.env` chứa dòng `API_KEY=<YOUR_API_KEY>` trên máy.
3. Chạy `git status`: `.env` không xuất hiện là đúng.

Kết quả cần đạt: File bí mật không bao giờ lọt vào commit.

20 CHƯƠNG 20

License, Releases và Tags

License cho biết người khác được làm gì với dự án. Releases đóng gói từng phiên bản để chia sẻ và tải về.

20.1 Chọn giấy phép (license)

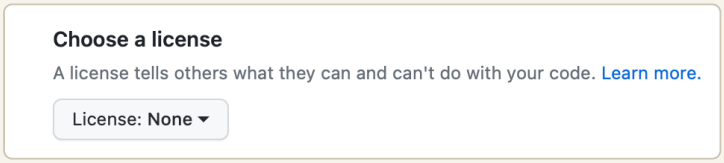
Bạn không bắt buộc phải có license. Nhưng không có license thì luật bản quyền mặc định áp dụng: bạn giữ mọi quyền, người khác **không được** sao chép, phân phối hay tạo sản phẩm phái sinh. Dự án mã nguồn mở nên có license; GitHub làm trang `choosealicense.com` để giúp chọn[80].

License	Tóm tắt dễ hiểu
MIT	Rất thoáng: ai cũng dùng, sửa, bán được, chỉ cần giữ thông báo bản quyền
Apache 2.0	Thoáng như MIT, thêm điều khoản về bằng sáng chế
GPL v3	Ai phân phối bản sửa đổi phải công bố mã nguồn theo cùng giấy phép
CC BY 4.0	Thường dùng cho nội dung, tài liệu (không phải phần mềm)

⚠ CHƯA CHẮC Bảng trên là tóm tắt phổ thông, không phải tư vấn pháp lý; đọc toàn văn trên choosealicense.com trước khi chọn.

20.2 Thêm license

1. Khi tạo repo: chọn ở mục **Choose a license**.
2. Repo có sẵn: **Add file** › **Create new file**, gõ tên `LICENSE`, bấm **Choose a license template**, chọn mẫu, commit.



Mục "Choose a license" khi tạo repository.

Nguồn: ảnh chính thức GitHub Docs, trang "Licensing a repository"[80]

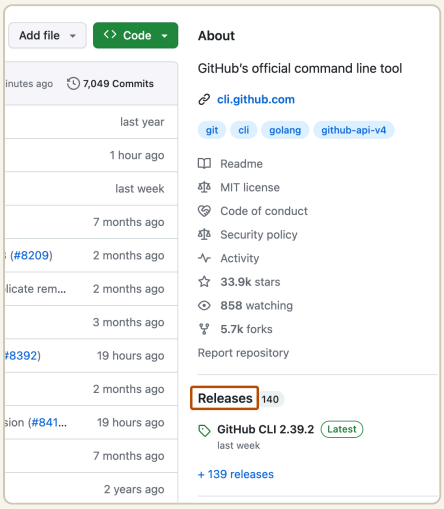
20.3 Tag và Release khác gì nhau?

Tag là nhãn đánh dấu một thời điểm trong lịch sử (ví dụ `v1.0.0`). **Release** xây trên một tag, thêm ghi chú phát hành và file đính kèm (binary) để người khác tải. GitHub tự gắn file zip và tarball mã nguồn tại thời điểm tag[81].

20.4 Tạo release

1. Trang repo › mục **Releases** bên phải › **Draft a new release**.
2. **Choose a tag**: gõ tag mới, ví dụ `v1.0.0`, chọn nhánh nguồn.
3. Đặt tiêu đề, viết ghi chú hoặc bấm **Generate release notes** để GitHub tự tạo từ các PR.
4. Kéo thả file đính kèm nếu có (bản PDF, file cài đặt).
5. Tích **This is a pre-release** nếu chưa ổn định. Bấm **Publish release** hoặc **Save draft**.

THEO DOCS "Managing releases in a repository"[82]. Repo bật *immutable releases* thì nên lưu nháp, đính kèm đủ file rồi mới publish.



Liên kết "Releases" trên trang chính của repo.
Nguồn: ảnh chính thức GitHub Docs, trang "Managing releases in a repository" [82]

ĐÁNH SỐ PHIÊN BẢN

Quy ước phổ biến **MAJOR.MINOR.PATCH**: `v1.0.0` bản đầu; `v1.1.0` thêm tính năng; `v1.1.1` sửa lỗi nhỏ; `v2.0.0` thay đổi lớn, có thể không tương thích bản cũ.

Xem tag

Trang **Releases** › tab **Tags**[83]. Trên máy: `git tag`, `git push origin v1.0.0`.

BÀI TẬP · PHIÊN BẢN ĐẦU TIÊN

1. Thêm license MIT cho `hello-world`.
2. Tạo release `v1.0.0`, bấm Generate release notes.
3. Đính kèm một file PDF giới thiệu (nếu có).

Kết quả cần đạt: Trang Releases có v1.0.0 với ghi chú và file tải về.

21 CHƯƠNG 21

GitHub Pages: trang web miễn phí

GitHub Pages biến repo thành trang web công khai, không cần thuê hosting. Chỉ cần vài cú bấm.

21.1 Hai loại trang

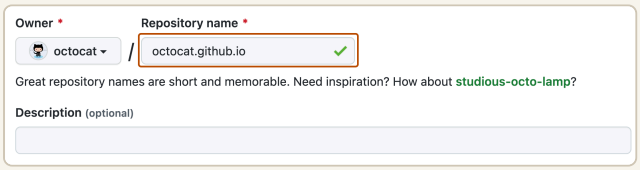
Loại	Địa chỉ	Ghi chú
Trang người dùng / tổ chức	`https://<owner>.github.io`	Repo phải tên `<owner>.github.io`; mỗi tài khoản một trang
Trang dự án	`https://<owner>.github.io/<repo>`	Bật cho bất kỳ repo nào

THEO DOCS "What is GitHub Pages?"[84]. Gói Free: Pages cho repo **public**; Pro, Team cho cả repo private[21].

21.2 Tạo trang cá nhân trong 10 phút

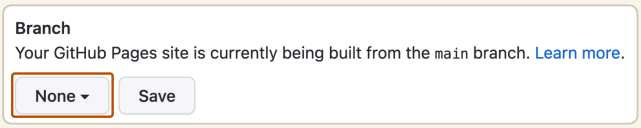
1. **+** › **New repository**, đặt tên `<username>.github.io` (thay bằng username của bạn).
2. Chọn chế độ hiển thị, bật **Add README**, bấm **Create repository**.
3. Vào **Settings** › mục **Pages** (nhóm "Code, planning, and automation").
4. Ở "Build and deployment", **Source** chọn **Deploy from a branch**; **Branch** chọn `main`, bấm **Save**.
5. Viết nội dung trong `README.md` (hoặc thêm `index.html`).
6. Mở `<username>.github.io`. Thay đổi có thể mất tới 10 phút mới hiện.

THEO DOCS "Quickstart for GitHub Pages"[85].



Đặt tên repo dạng octocat.github.io.

Nguồn: ảnh chính thức GitHub Docs, trang "Quickstart for GitHub Pages" [85]



Chọn nhánh làm nguồn xuất bản trong Settings › Pages.

Nguồn: ảnh chính thức GitHub Docs, trang "Quickstart for GitHub Pages" [85]

21.3 Đổi tiêu đề, giao diện

Trang mặc định dùng Jekyll. Tạo hoặc sửa file `_config.yml` ở gốc repo[85]:

_CONFIG.YML

```
theme: jekyll-theme-minimal
title: Trang của Minh
description: Ghi chú học GitHub và các dự án nhỏ
```

21.4 Hai cách xuất bản

Cách	Hợp với
Deploy from a branch	Trang tĩnh đơn giản: README, HTML, Jekyll
GitHub Actions	Trang dùng công cụ build (Astro, Vite, Hugo…) qua workflow tùy chỉnh

THEO DOCS "Configuring a publishing source"[86]. Tên miền riêng: thêm trong Settings › Pages › Custom domain và bật **Enforce HTTPS**.

21.5 Giới hạn cần biết

Hạng mục	Giới hạn
Kích thước trang đã xuất bản	tối đa 1 GB (repo nguồn khuyến nghị dưới 1 GB)
Băng thông	giới hạn mềm 100 GB/tháng
Số lần build	giới hạn mềm 10 lần/giờ (không áp dụng khi build bằng Actions tùy chỉnh)
Thời gian deploy	quá 10 phút sẽ hết thời gian chờ

KHÔNG ĐƯỢC DÙNG PAGES ĐỂ

Chạy kinh doanh trực tuyến, thương mại điện tử, SaaS, hay xử lý giao dịch nhạy cảm (mật khẩu, số thẻ). Pages cũng chịu Điều khoản dịch vụ của GitHub[87].

▶ VIDEO THAM KHẢO · TIẾNG ANH · QUÉT QR

Getting started with GitHub Pages for beginners | Tutorial

xem từ 00:55 đến 03:31 kênh **GitHub** · 48.218 lượt xem (17/10/2026) · đăng 13/04/2026

Đoạn: "Deploy from a branch" đến hết "Enforce HTTPS". Video chính thức: deploy từ nhánh, deploy bằng Actions, tên miền riêng, HTTPS.

youtu.be/b2r9Cdvssi0?t=55

BÀI TẬP · TRANG WEB CỦA TÔI

1. Tạo repo `<username>.github.io`, bật Pages từ nhánh main.
2. Viết README giới thiệu bản thân (dùng lại profile README).
3. Thêm `_config.yml` với theme và title.
4. Gửi link trang cho một người bạn.

Kết quả cần đạt: Trang `<username>.github.io` hoạt động.

22 CHƯƠNG 22

GitHub Actions cơ bản

Actions chạy các bước tự động mỗi khi có sự kiện trong repo, ví dụ mỗi lần push hay mở pull request.

22.1 Năm khái niệm

Khái niệm	Giải thích
Workflow	Quy trình tự động, viết bằng file YAML trong thư mục `.github/workflows`
Event	Sự kiện kích hoạt: push, mở PR, tạo issue, theo lịch...
Job	Nhóm các bước chạy trên cùng một máy (runner)
Step	Một lệnh shell hoặc một action
Action	Gói bước dựng sẵn, dùng lại được (ví dụ `actions/checkout`)
Runner	Máy chạy job: Ubuntu, Windows, macOS do GitHub cung cấp, hoặc máy tự host

THEO DOCS "Understanding GitHub Actions"[88].

22.2 Workflow đầu tiên

Tạo file `.github/workflows/github-actions-demo.yml` (đuôi bắt buộc `.yml` hoặc `.yaml`) với nội dung dưới đây, rồi commit[89]:

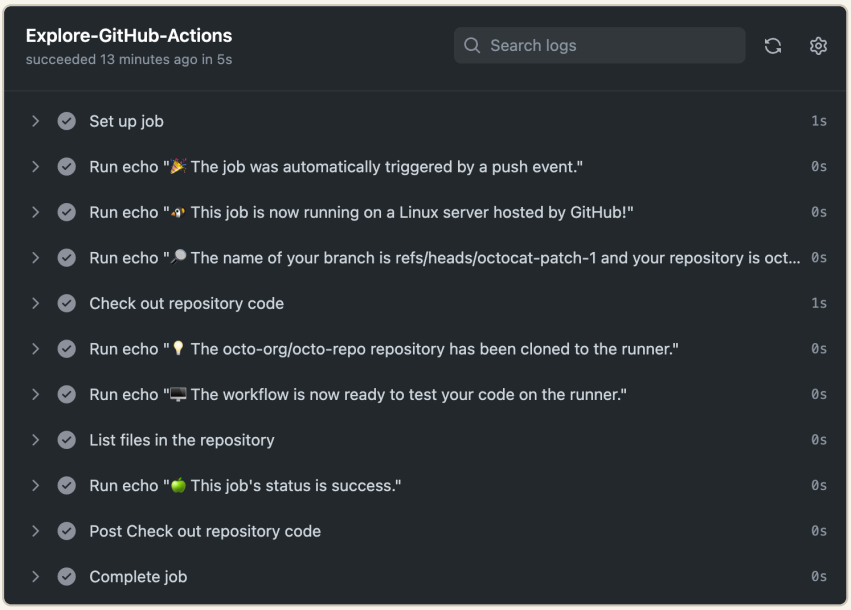
MẪU TỪ DOCS (RÚT GỌN, CHÚ THÍCH TIẾNG VIỆT)

```yaml
name: GitHub Actions Demo
run-name: ${{ github.actor }} is testing out GitHub Actions 🚀
on: [push]
jobs:
  Explore-GitHub-Actions:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Sự kiện kích hoạt: ${{ github.event_name }}"
      - run: echo "Đang chạy trên máy ${{ runner.os }} do GitHub cung cấp"
      - name: Check out repository code
        uses: actions/checkout@v6
      - name: List files in the repository
        run: |
          ls ${{ github.workspace }}
      - run: echo "Trạng thái job: ${{ job.status }}"
```

Mỗi lần push, workflow chạy trên máy Ubuntu mới tinh. Phần `uses: actions/checkout@v6` tải code repo về máy chạy.

22.3 Xem kết quả

1. Vào tab **Actions** của repo.
2. Chọn workflow "GitHub Actions Demo" ở thanh bên trái, bấm lần chạy mới nhất.
3. Bấm job **Explore-GitHub-Actions** để xem log từng bước. Dấu tích xanh là thành công, dấu X đỏ là lỗi.



Các bước đã chạy của workflow trong trang "Workflow run".
Nguồn: ảnh chính thức GitHub Docs, trang "Quickstart for GitHub Actions" [89]

22.4 Chi phí và bí mật

- Actions **miễn phí** với repo public dùng runner chuẩn và với runner tự host. Repo private dùng hạn mức theo gói: GitHub Free 2.000 phút/tháng, Pro 3.000 phút/tháng[45][21].
- Phút tính cho chủ repo, không phải người kích hoạt.
- Khóa API cho workflow: lưu ở **Settings** › **Secrets and variables** › **Actions**, gọi bằng `${{ secrets.TEN_KHOA }}`. Không bao giờ ghi thẳng khóa vào YAML[90].

THAY ĐỔI 2026 Từ 01/06/2026 Copilot code review tiêu tốn cả phút Actions (repo private)[46].

22.5 Ví dụ hữu ích cho người không code

Mục tiêu	Gợi ý
Kiểm tra chính tả, liên kết hỏng trong tài liệu	Tìm action trên GitHub Marketplace (từ khóa "link checker", "spell check")
Tự xuất bản trang web	Pages với nguồn GitHub Actions (chương 21)
Chào người mở issue đầu tiên	Workflow chạy khi có sự kiện `issues` hoặc `pull_request`

⚠ CHƯA CHẮC Action của bên thứ ba do cộng đồng viết: đọc mã, xem số sao và người duy trì trước khi dùng.

MẸO AN TOÀN VỚI ACTIONS

- Đặt `permissions` tối thiểu cho workflow khi đã quen.
- Dùng phiên bản action cố định (ví dụ `@v6`), dự án quan trọng có thể ghim theo SHA.
- Không in giá trị secret ra log.

▶ VIDEO THAM KHẢO · TIẾNG ANH · QUÉT QR

How to use GitHub Actions | GitHub for Beginners

xem từ 02:31 đến 07:26 kênh **GitHub** · 62.395 lượt xem (17/10/2026) · đăng 16/03/2026

Đoạn: "Building your first workflow" đến hết "How to test your workflow". Video chính thức "GitHub for Beginners": dựng workflow đầu tiên và kiểm tra kết quả.

youtu.be/BQrohJ3PT7I?t=151

BÀI TẬP · ROBOT ĐẦU TIÊN

1. Tạo workflow demo ở 22.2 trong `hello-world`.
2. Push một thay đổi nhỏ, xem tab Actions chạy.
3. Sửa YAML thêm một bước `run: echo "Xin chào từ Actions"`, push lại, tìm dòng đó trong log.

Kết quả cần đạt: Workflow chạy xanh, log có dòng chào của bạn.

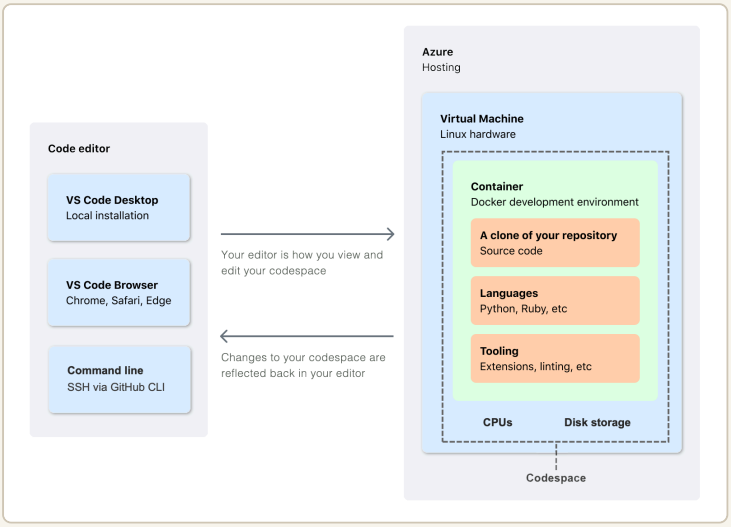
23 CHƯƠNG 23

GitHub Codespaces

Không muốn cài đặt gì trên máy? Codespaces cho bạn một "máy tính lập trình" chạy trên mây, mở ngay trong trình duyệt.

23.1 Codespaces là gì?

Codespace là môi trường phát triển chạy trên máy ảo do GitHub quản lý. Bạn dùng VS Code trên trình duyệt hoặc kết nối từ VS Code, JetBrains trên máy. Môi trường có thể cấu hình sẵn bằng **dev container**[91] [92].



Quan hệ giữa trình soạn thảo của bạn và codespace chạy trên máy ảo.

Nguồn: ảnh chính thức GitHub Docs, trang "What are GitHub Codespaces?" [91]

23.2 Hạn mức miễn phí

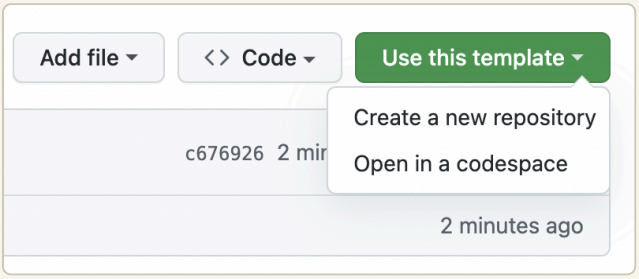
Gói tài khoản cá nhân	Lưu trữ / tháng	Core hours / tháng
GitHub Free	15 GB	120 giờ
GitHub Pro	20 GB	180 giờ

THEO DOCS "GitHub Codespaces billing"[93], "GitHub's plans"[21]. Core hours tính theo số nhân: máy 2 nhân chạy 1 giờ tính 2 core hours, nên 120 core hours tương đương 60 giờ trên máy 2 nhân. Trang pricing ghi giá vượt hạn mức từ $0.18/giờ tính toán và $0.07/GB lưu trữ mỗi tháng[94].

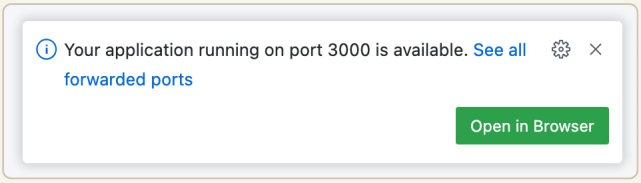
23.3 Mở codespace

1. Trên trang repo, bấm **Code** › tab **Codespaces** › **Create codespace on main**.
2. Chờ máy khởi động, VS Code mở trong tab trình duyệt.
3. Sửa file, mở Terminal (`Ctrl+``), chạy lệnh như trên máy thật.
4. Commit và push từ thanh **Source Control** bên trái.

THEO DOCS "Creating a codespace for a repository"[95], "Quickstart for GitHub Codespaces"[96].



Nút "Use this template" › "Open in a codespace".
Nguồn: ảnh chính thức GitHub Docs, trang "Quickstart for GitHub Codespaces" [96]



Thông báo ứng dụng đang chạy ở cổng 3000, mở được trong trình duyệt.
Nguồn: ảnh chính thức GitHub Docs, trang "Quickstart for GitHub Codespaces" [96]

23.4 Dừng để codespace chạy không

- Đóng tab trình duyệt **không** dừng codespace; nó chạy tiếp tới khi hết thời gian chờ không hoạt động[97].
- Dừng thủ công: vào `github.com/codespaces`, bấm **…** cạnh codespace › **Stop codespace**.
- Docs ghi codespace có thời gian sống tối đa 12 giờ bất kể cài đặt thời gian chờ.
- Xóa codespace không dùng nữa để giải phóng dung lượng lưu trữ (dung lượng vẫn tính khi đã dừng).

▶ VIDEO THAM KHẢO · TIẾNG ANH · QUÉT QR

How to use GitHub Codespaces for Coding and Data Science

xem từ 03:17 đến 06:08 kênh **Data Professor** · 77.968 lượt xem (17/10/2026) · đăng 21/08/2023
Đoạn: "Creating a new code space" + "Setting up the environment". Data Professor: tạo codespace và thiết lập môi trường từng bước.
youtu.be/kvJf8s18Vr4?t=197

BÀI TẬP · LẬP TRÌNH TRÊN MÂY

1. Mở codespace từ repo `hello-world`.
2. Sửa README trong codespace, commit và push.
3. Dừng codespace ở `github.com/codespaces`.

Kết quả cần đạt: Biết mở, dùng, dừng một codespace.

24 CHƯƠNG 24

GitHub Copilot

Copilot là trợ lý AI của GitHub: giải thích code, viết code, review PR, và nhận việc như một đồng đội. Chương này cập nhật theo docs tháng 10/2026.

24.1 Copilot có ở đâu?

Nơi dùng	Dùng để
Trang web GitHub	Hỏi về repo, issue, PR; giao việc cho agent. Không cần cài gì, docs khuyên bắt đầu ở đây.
IDE (VS Code, Visual Studio, JetBrains, Eclipse, Xcode)	Gợi ý code trong lúc gõ, chat về code, agent sửa file giúp bạn.
GitHub Copilot app (máy tính)	Quản lý nhiều việc do agent làm song song, nhiều repo, trọn vòng pull request.
GitHub Copilot CLI	Làm việc với Copilot trong Terminal.
GitHub Mobile	Chat với Copilot khi không ngồi máy tính.
GitHub Desktop	Gợi ý lời nhắn commit và tóm tắt.

THEO DOCS "Where to use GitHub Copilot"[98]. Xem tính năng tài khoản bạn đang có tại `github.com/settings/copilot/features`.

24.2 Bốn kiểu giúp việc trên GitHub.com

Kiểu	Làm gì
Chat	Hỏi đáp về repo, giải thích code, gợi ý cách sửa
Agentic (Copilot cloud agent)	Nghiên cứu repo, lập kế hoạch, sửa code, chạy test trong môi trường đám mây tạm thời, tạo nhánh, commit, PR
Code review	Nhận xét PR, gợi ý sửa
Automations	Chạy việc lặp lại theo lịch hoặc theo sự kiện của repo

THEO DOCS "GitHub Copilot on GitHub.com"[99]. THAY ĐỔI 2026 Tên "Copilot coding agent" trong video, bài viết cũ nay docs gọi là **Copilot cloud agent**.

▶ VIDEO THAM KHẢO · TIẾNG ANH · QUÉT QR

How to use GitHub Copilot (the complete beginner's guide)

xem từ 05:35 đến 08:14 kênh **GitHub** · 195.970 lượt xem (17/10/2026) · đăng 30/07/2025

Đoạn: phần "Getting started": gợi ý code và Copilot Chat. Video chính thức: cài Copilot trong VS Code, gợi ý code và Copilot Chat. Phần gói cước trong video (07/2025) đã cũ, xem 24.3.

youtu.be/SJqGYwRq0uc?t=335

24.3 Các gói Copilot THAY ĐỔI 2026

Gói	Giá (USD)	AI credits mỗi tháng	Ghi chú chính
Copilot Free	Miễn phí	Một hạn mức AI credits	Chỉ tự động chọn model; 2.000 lượt gợi ý code/tháng; agent giới hạn
Copilot Student	Miễn phí (sinh viên đã xác minh)	Một hạn mức AI credits	Gợi ý code không giới hạn; không có agent bên thứ ba
Copilot Pro	$10/tháng (miễn phí cho một số người)	1.000 cơ bản + 500 flex = 1.500	Một số model; có cloud agent
Copilot Pro+	$39/tháng	3.900 + 3.100 = 7.000	Model cao cấp
Copilot Max	$100/tháng	10.000 + 10.000 = 20.000	Ưu tiên model cao cấp
Copilot Business	$19/người/tháng	1.900 mỗi người (gộp chung)	Cho tổ chức
Copilot Enterprise	$39/người/tháng	3.900 mỗi người (gộp chung)	Cho doanh nghiệp

THEO DOCS "Plans for GitHub Copilot"[47], "Usage-based billing for individuals"[100], gói tổ chức[101]. Mọi gói đều có Copilot CLI và Copilot app. Copilot Free chỉ dành cho người không được cấp Copilot qua tổ chức. Giáo viên đã xác minh và người duy trì dự án mã nguồn mở phổ biến có thể được Pro miễn phí.

THAY ĐỔI LỚN TỪ 01/06/2026: TÍNH PHÍ THEO MỨC DÙNG

- Thay "premium requests" bằng **GitHub AI Credits**: 1 AI credit = 0,01 USD; mức tiêu hao tính theo token vào, ra, cache và model[102][46].
- Gói trả phí gồm credits cơ bản (bằng giá gói) dùng trước, sau đó tới phần flex. Credits không cộng dồn sang tháng sau, đặt lại lúc 00:00 UTC (07:00 giờ Việt Nam) ngày 1 hằng tháng[100].
- Gợi ý code và next edit suggestions **không** tính credits, không giới hạn với gói trả phí. Chat, CLI, cloud agent, Spaces, Spark, agent bên thứ ba thì tính credits.
- Dùng hết: đặt ngân sách thêm (budget) bằng USD hoặc nâng gói. Người đang trả **theo năm** Pro/Pro+ giữ cách tính premium requests cũ tới khi hết hạn[102].

24.4 Tính năng theo gói cá nhân

Tính năng	Free	Student	Pro	Pro+	Max
Copilot cloud agent	Không	Có	Có	Có	Có
Agent mode (IDE)	Có	Có	Có	Có	Có
Copilot code review	Chỉ "Review selection" trong VS Code	Có	Có	Có	Có
MCP	Có	Có	Có	Có	Có
Agent bên thứ ba (preview)	Không	Không	Có	Có	Có

THEO DOCS Bảng "Agents" trong "Plans for GitHub Copilot"[47].

24.5 Bắt đầu với Copilot Free

1. Đăng nhập GitHub, mở trang cài đặt Copilot `github.com/settings/copilot`.
2. Làm theo hướng dẫn để kích hoạt gói Free (hoặc Student nếu đã xác minh qua GitHub Education).
3. Thử ngay trên web: mở một file trong repo bất kỳ, bấm biểu tượng Copilot góc phải trên, đặt câu hỏi.
4. Muốn dùng trong VS Code: cài tiện ích GitHub Copilot, đăng nhập bằng tài khoản GitHub.

⚠ CHƯA CHẮC Trang hướng dẫn riêng về kích hoạt Copilot Free trên docs trả lỗi 404 khi biên soạn (17/10/2026); các bước 1–2 suy ra từ trang Student[4] và quickstart[103]. Nếu chỉ thấy trang thanh toán, đừng mua; đợi vài ngày hoặc liên hệ GitHub Support.

24.6 Hỏi Copilot trên GitHub.com

CÂU HỎI MẪU — COPILOT CHAT TRÊN WEB

Giải thích file này làm gì, bằng tiếng Việt, cho người không biết lập trình.
Thay đổi gần nhất của file này là gì và vì sao?
Tóm tắt pull request này trong 5 gạch đầu dòng.
Những test nào nên có cho hàm này?

THEO DOCS Câu hỏi gợi ý theo "Quickstart for using GitHub Copilot on GitHub.com"[103] và "Asking GitHub Copilot questions in GitHub"[104].

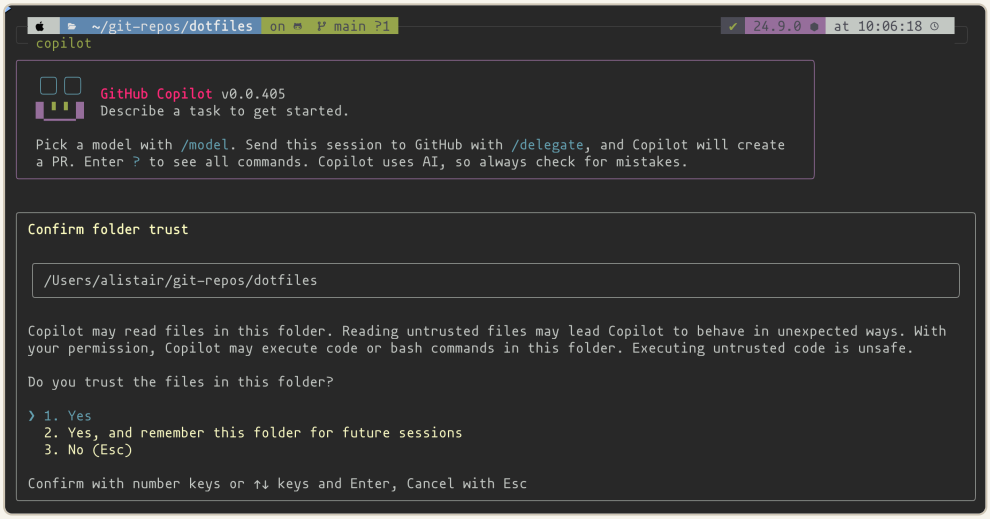
24.7 Giao issue cho Copilot cloud agent

Cần gói trả phí (ví dụ Copilot Pro) hoặc Student. Copilot đọc tiêu đề, mô tả, bình luận của issue, làm việc trong môi trường đám mây, rồi mở pull request và yêu cầu bạn review[103].

1. Bật Copilot cloud agent trong cài đặt Copilot cho repo cá nhân của bạn.
2. Mở một issue (hoặc tạo mới), mô tả rõ yêu cầu và tiêu chí hoàn thành.
3. Thanh bên phải › **Assignees** › chọn **Copilot**; có thể thêm ghi chú ở ô **Optional prompt**.
4. Chờ Copilot mở PR. Xem diff, xem session log.
5. Muốn sửa thêm: bình luận `@copilot` trong PR kèm yêu cầu. Hài lòng thì merge.

ISSUE VIẾT TỐT CHO AGENT — COPILOT CLOUD AGENT

Tiêu đề: Thêm trang Liên hệ cho website
Mô tả: Tạo file `lien-he.html` theo cùng kiểu với `index.html`, có tên, email `<email-cong-khai>` và link Facebook. Thêm liên kết "Liên hệ" vào menu của `index.html`.
Hoàn thành khi: trang mở được trên GitHub Pages, menu có mục mới, không đổi nội dung khác.



Màn hình chào của Copilot CLI trong Terminal.

Nguồn: ảnh chính thức GitHub Docs, trang "GitHub Copilot CLI" [105]

THEO DOCS Agent có thể bắt đầu từ bảng agents, từ cuộc chat, hoặc từ issue/PR; session log cho thấy agent đã làm gì, nhưng không thay được việc bạn tự review, kiểm thử[99].

▶ VIDEO THAM KHẢO · TIẾNG ANH · QUÉT QR

How the GitHub Copilot coding agent works | GitHub Checkout

xem từ 00:37 đến 02:36 kênh **GitHub** · 75.042 lượt xem (17/10/2026) · đăng 30/05/2025

Đoạn: "Assigning multiple issues to Copilot" + "Reviewing draft pull requests". Video chính thức (05/2025, khi còn tên "coding agent"): giao nhiều issue cho Copilot và review PR nháp do Copilot tạo.

youtu.be/1GVBRhDI5No?t=37

24.8 Copilot CLI và Copilot app

Công cụ	Cài đặt và ghi chú
Copilot CLI	`npm install -g @github/copilot` (Node.js 22+), hoặc `winget install GitHub.Copilot` (Windows), `brew install --cask copilot-cli` (macOS, Linux). Cần gói Copilot đang hoạt động; Windows cần PowerShell 6+ [106].
GitHub Copilot app	Ứng dụng máy tính cho macOS, Windows, Linux, GA ngày 17/06/2026. Mở phiên làm việc từ issue, PR hoặc prompt; chạy song song nhiều phiên, mỗi phiên một nhánh và worktree; xem diff, terminal, trình duyệt tích hợp rồi mở PR[107].

THAY ĐỔI 2026 Tháng 10/2026: "computer use" (Copilot điều khiển ứng dụng máy tính, luôn xin phép trước) ở dạng public preview trong Copilot CLI và Copilot app trên macOS, Windows[108]; local sandboxing (giới hạn quyền của agent với file, mạng, thông tin đăng nhập) đã GA, không tốn thêm phí[109][110].

24.9 Copilot code review

Trong PR, thêm **Copilot** vào **Reviewers**. Copilot đọc thay đổi, thu thập ngữ cảnh trong repo, nêu vấn đề và gợi ý sửa. Có thể cấu hình review tự động[111][99]. Mỗi lần review tốn AI credits và phút Actions với repo private[45].

24.10 Dùng Copilot an toàn, hiệu quả

- **Luôn tự review** code, nội dung do AI tạo. AI có thể sai, và câu trả lời mỗi lần mỗi khác.
- Đưa ngữ cảnh rõ: file nào, mục tiêu gì, ràng buộc gì, tiêu chí hoàn thành.
- Không dán mật khẩu, token, dữ liệu khách hàng vào prompt.
- Copilot có thể sinh code trùng code công khai; docs có tính năng tham chiếu mã khớp kèm thông tin giấy phép để bạn quyết định[99].
- Theo dõi mức dùng AI credits trong trang cài đặt để không bất ngờ.

PROMPT HỎI LỖI GIT — COPILOT CHAT / CLI

Bạn là người hướng dẫn GitHub kiên nhẫn. Giải thích lỗi sau bằng tiếng Việt đơn giản, nêu nguyên nhân có thể và các bước sửa từng bước: <dán thông báo lỗi, đã xóa thông tin nhạy cảm>

BÀI TẬP · TRỢ LÝ AI

1. Kích hoạt Copilot Free.
2. Mở một file trong repo công khai bất kỳ, hỏi Copilot "Giải thích file này".
3. Nhờ Copilot viết README cho `hello-world`, tự sửa lại cho đúng ý bạn.
4. (Gói trả phí) Giao một issue nhỏ cho Copilot, review PR nó tạo.

Kết quả cần đạt: Bạn dùng được Copilot để hiểu và viết nội dung, và biết tự kiểm tra kết quả.

25 CHƯƠNG 25

Bảo mật repository

GitHub có sẵn nhiều "camera an ninh" miễn phí cho repo public. Bật lên và biết đọc cảnh báo là đủ an toàn cho người mới.

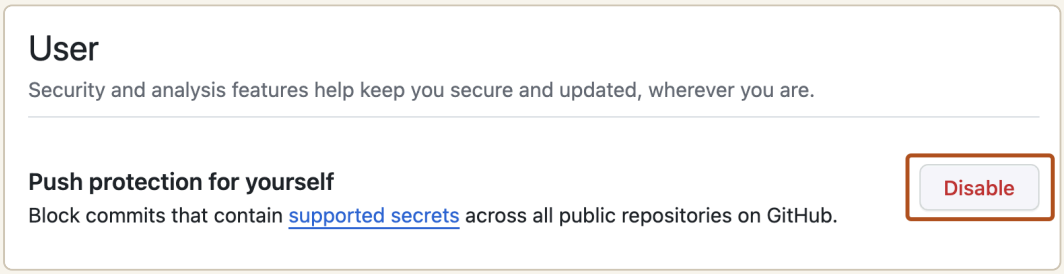
25.1 Bức tranh chung

Tính năng	Làm gì	Repo public
Dependabot alerts	Báo khi thư viện bạn dùng có lỗ hổng đã biết	Có trong gói Free
Dependabot security / version updates	Tự mở PR nâng cấp thư viện (version updates cấu hình bằng `dependabot.yml`)	Có
Secret scanning	Quét toàn bộ lịch sử, issue, PR, discussion tìm khóa, mật khẩu bị lộ	Tự động, miễn phí
Push protection	Chặn push khi phát hiện khóa trong code	Bật mặc định
Code scanning (CodeQL) + Copilot Autofix	Tìm lỗ hổng trong code, gợi ý bản sửa	Miễn phí cho repo public
Security policy (`SECURITY.md`)	Hướng dẫn người khác báo lỗ hổng	Mọi repo

THEO DOCS "GitHub security features"[112], "About Dependabot alerts"[113], "About Dependabot version updates"[114], "Secret scanning"[115], "About code scanning"[116]. Repo private của tổ chức cần mua GitHub Secret Protection / Code Security (Team, Enterprise Cloud).

25.2 Push protection cho chính bạn

"Push protection for users" bật sẵn, chặn bạn lỡ đẩy khóa bí mật lên **mọi repo public**, kể cả khi repo đó không bật secret scanning[112]. Kiểm tra tại **Settings** › **Code security** › mục **User**[117].



Mục "Push protection for yourself" trong phần User của trang Code security.
Nguồn: ảnh chính thức GitHub Docs, trang "Managing push protection for users" [117]

25.3 Khi bị chặn push vì lộ khóa

1. Đọc thông báo: GitHub cho biết file, dòng, loại khóa.
2. Xóa khóa khỏi code, chuyển sang biến môi trường hoặc GitHub Secrets.
3. Nếu khóa đã nằm trong commit chưa push: sửa commit (ví dụ `git commit --amend` hoặc commit lại) rồi push.
4. Nếu khóa từng lên GitHub: **thu hồi khóa ngay** tại dịch vụ cấp khóa, tạo khóa mới.

THEO DOCS "About push protection"[118]. THAY ĐỔI 2026 13/10/2026: secret scanning chặn mặc định khóa Anthropic, thêm Salesforce[119]; GitHub liên tục bổ sung loại khóa mới.

25.4 Dependabot alerts

- Xem ở tab **Security and quality** (tên tab mới trong docs) › **Dependabot**.
- Mỗi cảnh báo nêu thư viện, mức nghiêm trọng, phiên bản đã vá.
- Có thể giao cảnh báo cho người, nhóm hoặc agent (Copilot, agent bên thứ ba): agent tự mở PR nháp đề xuất bản sửa[113].
- Bật Dependabot security updates để GitHub tự mở PR nâng cấp.

25.5 Thêm SECURITY.md

SECURITY.MD MẪU

```
# Chính sách bảo mật
## Phiên bản được hỗ trợ
Bản mới nhất trên nhánh main.
## Báo lỗ hổng
Vui lòng KHÔNG tạo issue công khai. Dùng mục "Report a vulnerability"
trong tab Security hoặc gửi email tới <email-bao-mat>.
```

THEO DOCS "Adding a security policy to your repository"[120].

▶ VIDEO THAM KHẢO · TIẾNG ANH · QUÉT QR

Getting started with GitHub security | GitHub for Beginners

xem từ 02:36 đến 04:37 kênh **GitHub** · 28.871 lượt xem (17/10/2026) · đăng 30/03/2026

Đoạn: "What is Dependabot?" + "Fixing vulnerabilities with Copilot Autofix". Video chính thức: Dependabot là gì và sửa lỗ hổng với Copilot Autofix.

youtu.be/zhxXaFzzJYA?t=156

BÀI TẬP · BẬT AN NINH

1. Mở tab Security của `hello-world`, bật Dependabot alerts và security updates.
2. Thêm `SECURITY.md` theo mẫu.
3. Kiểm tra Push protection for yourself đang bật.

Kết quả cần đạt: Repo có chính sách bảo mật, cảnh báo đã bật.

26 CHƯƠNG 26

Gists

Gist là cách nhanh nhất để chia sẻ một đoạn ghi chú hay code mà không cần tạo repo.

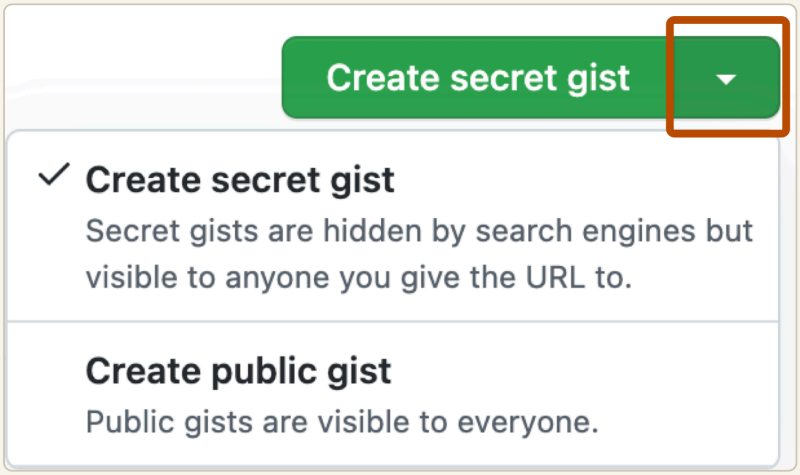
26.1 Public gist và secret gist

	Public gist	Secret gist
Hiện ở trang Discover, tìm kiếm được	Có	Không
Ai có link xem được	Mọi người	Mọi người có link (secret không có nghĩa là riêng tư)
Phù hợp	Chia sẻ mẹo, đoạn code mẫu	Gửi nháp cho vài người

THEO DOCS "Creating gists"[121]. Cần thật sự riêng tư thì dùng repo private. GitHub quét secret gist để báo đối tác khi phát hiện khóa bị lộ.

26.2 Tạo gist

1. Vào `gist.github.com` (đã đăng nhập).
2. Nhập mô tả, tên file kèm đuôi (ví dụ `ghi-chu.md`) để có tô màu cú pháp.
3. Viết nội dung; bấm **Add file** nếu cần nhiều file.
4. Chọn **Create secret gist** hoặc **Create public gist** ở menu bên cạnh nút tạo.



Menu chọn chế độ hiển thị khi tạo gist mới.
Nguồn: ảnh chính thức GitHub Docs, trang "Creating gists" [121]

Mỗi gist thực chất là một repo Git nhỏ: có lịch sử sửa đổi, fork, gắn sao, bình luận được. BÀI TẬP Tạo một secret gist chứa bảng lệnh Git bạn hay quên, gửi link cho chính mình.

27 CHƯƠNG 27

Organizations và Teams

Khi nhiều người cùng sở hữu dự án, hãy dùng tài khoản tổ chức thay vì để mọi thứ trong tài khoản cá nhân.

27.1 Vì sao cần organization?

- Repo thuộc về nhóm, không phụ thuộc một người (người rời đi, repo vẫn còn).
- Phân quyền chi tiết theo nhóm (team) và theo repo.
- Owner (chủ sở hữu) quản lý thành viên, cài đặt bảo mật, thanh toán[8].
- Gói GitHub Free cho tổ chức: không giới hạn repo public và private (repo private bị giới hạn tính năng), 2.000 phút Actions/tháng[21].

27.2 Tạo organization

1. Ảnh đại diện › **Settings** › mục "Access" › **Organizations**.
2. Bấm **New organization**, chọn gói (Free để bắt đầu).
3. Đặt tên tổ chức, email liên hệ, mời thành viên.

THEO DOCS "Creating a new organization from scratch"[122].

27.3 Teams

Đặc điểm	Chi tiết
Quyền theo team	Owner và team maintainer cấp quyền admin, write, read trên repo cho cả team
Nhắc cả team	`@ten-to-chuc/ten-team` gửi thông báo tới mọi thành viên
Visible / Secret	Visible: mọi thành viên tổ chức thấy; Secret: chỉ thành viên team và owner thấy
Nested teams	Team con thừa hưởng quyền của team cha

THEO DOCS "About teams"[123]. ⚠ CHƯA CHẮC Danh sách đầy đủ vai trò trong tổ chức (moderator, billing manager, security manager…) xem trang "Roles in an organization" trên docs.

> GỢI Ý CHO LỚP HỌC
>
> Tạo tổ chức `lop-12a-2026`; mỗi nhóm một team; mỗi đồ án một repo private; giáo viên là owner. Học sinh dùng Issues và Projects để chia việc, nộp bài bằng pull request.

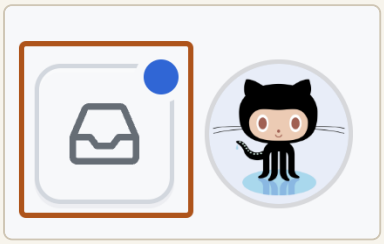
28 CHƯƠNG 28

Thông báo và bảng điều khiển

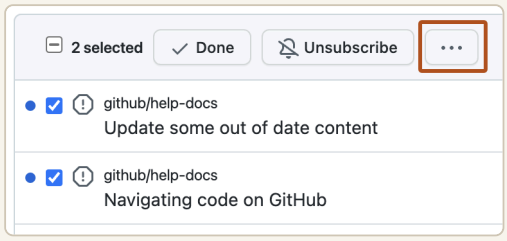
Theo dõi đúng việc, bỏ qua tiếng ồn. Ai tham gia nhiều repo đều cần một hộp thư thông báo gọn gàng.

28.1 Hộp thư thông báo (Inbox)

Biểu tượng hộp thư góc phải trên có chấm xanh khi có thông báo chưa đọc. Inbox hiện mọi thông báo bạn chưa hủy theo dõi hoặc chưa đánh dấu xong[124].



Biểu tượng hộp thư có chấm xanh: có thông báo chưa đọc.
Nguồn: ảnh chính thức GitHub Docs, trang "Managing notifications from your inbox" [124]



Menu xử lý nhiều thông báo cùng lúc.
Nguồn: ảnh chính thức GitHub Docs, trang "Managing notifications from your inbox" [124]

Thao tác	Tác dụng
Done	Đánh dấu xong, đưa khỏi inbox (xem lại bằng bộ lọc `is:done`)
Save	Lưu để xem sau, giữ vô thời hạn
Unsubscribe	Hủy theo dõi cuộc trao đổi cho tới khi bạn được nhắc tên hoặc được yêu cầu review
Bộ lọc tùy chỉnh	Ví dụ chỉ xem PR chờ bạn review, issue nhắc tên bạn

28.2 Bạn được theo dõi khi nào?

- Tự động theo dõi mọi repo bạn tạo trong tài khoản cá nhân.
- Khi bạn bình luận, được nhắc tên, được giao việc, được yêu cầu review.
- Khi bạn bấm Watch repo hoặc Subscribe một issue, PR[58].
- Chọn nhận qua web, email hay GitHub Mobile ở **Settings** › **Notifications**.

28.3 Dashboard mới THAY ĐỔI 2026

Từ 01/10/2026, giao diện dashboard mới (trang chủ khi đăng nhập) là mặc định cho mọi người: xem phiên làm việc của agent, issue, PR ở một nơi; mỗi mục tối đa 12 dòng, có bộ lọc; bảng tin tách sang tab **Feed**; bắt đầu việc ngay từ dashboard như giao issue cho Copilot. Muốn quay lại giao diện cũ, bấm menu cạnh chữ "Preview"[125].

28.4 Thay đổi về email thông báo THAY ĐỔI 2026

Từ 15/10/2026, email thông báo cho sự kiện push của repo đã bị ngừng để chống spam. Ai cần email khi có push có thể tự dựng workflow Actions gửi qua máy chủ email riêng[126].

28.5 Công thức giảm "tiếng ồn"

1. Mỗi tuần vào trang **Watching**, bỏ theo dõi repo không còn quan tâm.
2. Với repo lớn, chọn Watch › **Custom** chỉ nhận Releases.
3. Dùng bộ lọc "Review requested" và "Assigned" để làm việc quan trọng trước.
4. Đánh dấu Done ngay khi đọc xong; Save khi cần làm sau.

BÀI TẬP · INBOX GỌN

1. Mở Inbox, thử Done, Save, Unsubscribe mỗi loại một lần.
2. Tạo một bộ lọc tùy chỉnh.
3. Đặt Watch › Custom › Releases cho một repo bạn đã gắn sao.

Kết quả cần đạt: Inbox chỉ còn thông báo cần xử lý.

29 CHƯƠNG 29

GitHub Mobile

Ứng dụng di động chính thức giúp bạn xử lý thông báo, review PR, xác thực đăng nhập và hỏi Copilot ở bất cứ đâu.

29.1 Làm được gì trên GitHub Mobile?

- Quản lý, phân loại, dọn thông báo.
- Đọc, review, cộng tác trên issue và pull request; sửa file trong PR.
- Tìm kiếm, duyệt người dùng, repo, tổ chức; tìm code trong một repo.
- Nhận thông báo đẩy khi có người nhắc tên bạn.
- Dùng làm phương thức xác thực hai lớp, xác nhận đăng nhập trên thiết bị lạ.
- Hỏi Copilot Chat các câu hỏi về lập trình.

THEO DOCS "GitHub Mobile"[127].

29.2 Cài đặt và bật xác thực bằng Mobile

1. Tải "GitHub" từ App Store hoặc Google Play (nhà phát triển: GitHub).
2. Đăng nhập bằng tài khoản của bạn.
3. Sau khi đã có TOTP hoặc SMS, GitHub Mobile có thể dùng làm phương thức 2FA: khi đăng nhập trên máy tính, điện thoại hiện yêu cầu xác nhận[13].

CẢNH GIÁC

Chỉ bấm chấp thuận khi **chính bạn** đang đăng nhập. Yêu cầu xác nhận bất ngờ có thể là dấu hiệu ai đó biết mật khẩu của bạn: từ chối và đổi mật khẩu ngay.

BÀI TẬP · DI ĐỘNG

1. Cài GitHub Mobile, đăng nhập.
2. Bật thông báo đẩy cho việc được nhắc tên và review.
3. Mở một PR trên điện thoại, để lại một bình luận.

Kết quả cần đạt: Bạn xử lý được thông báo khi không ngồi máy tính.

30 CHƯƠNG 30

Gói dịch vụ và chi phí

Hầu hết người mới chỉ cần gói Free. Biết rõ cái gì miễn phí, cái gì tính tiền giúp bạn không bị bất ngờ.

30.1 Giá niêm yết trên github.com/pricing (17/10/2026)

Gói	Giá niêm yết	Ghi chú
Free	$0, mãi mãi	Cá nhân và tổ chức
Team	$4/người/tháng	Trang ghi "cho 12 tháng đầu*"
Enterprise	Từ $21/người/tháng	Trang ghi "cho 12 tháng đầu*", có dùng thử miễn phí

Nguồn: [94]. ⚠ CHƯA CHẮC Chú thích dấu * (điều kiện sau 12 tháng) không đọc được khi biên soạn; trang pricing không ghi giá GitHub Pro cá nhân. Kiểm tra trực tiếp trước khi mua.

30.2 Mỗi gói có gì (theo docs)

	Free (cá nhân)	Pro (cá nhân)	Team (tổ chức)
Repo	Không giới hạn public và private (private giới hạn tính năng)	Như Free + công cụ nâng cao cho repo private	Như Free tổ chức + công cụ nâng cao
Actions (repo private)	2.000 phút/tháng	3.000 phút/tháng	3.000 phút/tháng
Packages	500 MB	2 GB	2 GB
Codespaces	120 core hours, 15 GB	180 core hours, 20 GB	Trả theo mức dùng
Hỗ trợ	Community	Email	Email
Repo private: nhánh bảo vệ, nhiều reviewer, code owners, Pages, Wiki	Không	Có	Có

THEO DOCS "GitHub's plans"[21]. Repo public dùng Actions runner chuẩn miễn phí[45]. Team có thể mua thêm GitHub Code Security, Secret Protection, Code Quality.

30.3 Những khoản tính theo mức dùng

Dịch vụ	Miễn phí tới	Vượt mức
Actions	Repo public: miễn phí; private: theo hạn mức gói	Tính theo phút, theo loại máy
Codespaces	Hạn mức core hours, lưu trữ của tài khoản cá nhân	Từ $0.18/giờ tính toán, $0.07/GB/tháng lưu trữ[94]
Copilot	Copilot Free; credits đi kèm gói	AI credits 0,01 USD/credit, cần đặt budget
Packages	500 MB (Free)	Tính theo GB

MẶC ĐỊNH AN TOÀN

Trang pricing ghi Codespaces "$0 spend limit" cho gói Free: chưa đặt hạn mức chi tiêu và phương thức thanh toán thì không phát sinh phí vượt mức. Kiểm tra mục **Settings** › **Billing and licensing** để xem mức dùng và đặt ngân sách.

⚠ CHƯA CHẮC Tên mục Billing có thể khác chút theo giao diện.

30.4 Những thay đổi giá, gói đáng chú ý năm 2026 THAY ĐỔI 2026

Thời điểm	Thay đổi
27/04/2026	GitHub công bố Copilot chuyển sang tính phí theo mức dùng (AI credits)[102]
01/06/2026	AI credits áp dụng cho mọi gói Copilot; Copilot code review tiêu tốn thêm phút Actions[46]
06/2026	Copilot Business, Enterprise hiện hữu được credits khuyến mãi 3 tháng (06–08/2026)[101]
Hiện tại	Có thêm gói Copilot Max $100/tháng và Copilot Student miễn phí trong bảng gói[47]

⚠ CHƯA CHẮC Sách không có dữ liệu chính thức về thời điểm ra mắt chính xác của Copilot Max và Copilot Student; chỉ xác nhận chúng có trong docs hiện tại.

LỜI KHUYÊN CHỌN GÓI

- Mới học, làm dự án cá nhân, mã nguồn mở: **Free** là đủ.
- Cần nhánh bảo vệ, Pages cho repo private: cân nhắc **Pro**.
- Nhóm, công ty cần quản trị: **Team**. Doanh nghiệp lớn, tuân thủ: **Enterprise**.
- Sinh viên: đăng ký GitHub Education để nhận Copilot Student và các quyền lợi khác.

31 CHƯƠNG 31

Thói quen tốt

Những thói quen nhỏ, lặp lại mỗi ngày, quyết định bạn làm việc trên GitHub chuyên nghiệp hay không.

#	Thói quen	Vì sao
1	Repo nào cũng có README, license (nếu công khai) và .gitignore	Người khác hiểu, được phép dùng, repo không rác
2	Không commit mật khẩu, token, file .env	Lộ khóa rất khó thu hồi khỏi lịch sử
3	Bật 2FA, passkey; cất mã khôi phục	Bảo vệ toàn bộ tài sản trên GitHub
4	Làm trên nhánh, không sửa thẳng main khi làm nhóm	Main luôn ở trạng thái chạy được
5	Commit nhỏ, lời nhắn rõ ràng	Dễ review, dễ quay lại
6	PR nhỏ, mô tả đủ "gì, vì sao, kiểm tra thế nào"	Review nhanh, ít xung đột
7	Liên kết PR với issue bằng Closes #số	Theo dõi tiến độ tự động
8	Pull trước khi bắt đầu làm	Tránh xung đột
9	Xóa nhánh đã merge	Repo gọn
10	Bảo vệ nhánh main trong repo nhóm	Ngăn đẩy nhầm, buộc review
11	Dùng nhãn, milestone, Projects nhất quán	Cả nhóm nhìn cùng một bức tranh
12	Bật Dependabot và đọc cảnh báo	Phát hiện lỗ hổng sớm
13	Gắn phiên bản bằng Releases, ghi chú thay đổi	Người dùng biết có gì mới
14	Review kỹ mọi thứ AI tạo trước khi merge	AI có thể sai, có thể tạo lỗ hổng
15	Dọn thông báo, bỏ Watch repo không cần	Không bỏ lỡ việc quan trọng

Các thói quen tổng hợp từ GitHub flow[5], "About pull requests"[39] và các chương trước.

32 CHƯƠNG 32

Xử lý sự cố thường gặp

Lỗi là chuyện bình thường. Phần lớn lỗi Git và GitHub thuộc vài nhóm quen thuộc dưới đây.

Triệu chứng	Nguyên nhân thường gặp	Cách xử lý
`Authentication failed` / hỏi mật khẩu khi push	GitHub không nhận mật khẩu tài khoản qua HTTPS	Đăng nhập bằng `gh auth login` hoặc Git Credential Manager; hoặc dùng SSH (16.2)
`Permission denied (publickey)`	Chưa thêm SSH key vào GitHub, hoặc sai key	Kiểm tra `ssh -T git@github.com`; thêm khóa công khai vào Settings › SSH and GPG keys
`rejected ... (fetch first)` khi push	Trên GitHub có commit mới mà máy chưa có	`git pull`, giải xung đột nếu có, rồi push lại
`CONFLICT (content)`	Hai nhánh sửa cùng dòng	Làm theo chương 11
Push bị chặn vì "secret"	Push protection phát hiện khóa	Xóa khóa khỏi commit, thu hồi khóa (25.3)
File trên 100 MiB bị từ chối	Vượt giới hạn kích thước	Bỏ file khỏi commit; dùng Git LFS hoặc Releases
Trang Pages báo 404	Sai tên repo, chưa chọn nguồn, chưa có index/README, chờ chưa đủ	Kiểm tra Settings › Pages; tên `<username>.github.io`; chờ tới 10 phút
Workflow Actions không chạy	File không nằm trong `.github/workflows` hoặc sai đuôi, sai YAML	Kiểm tra đường dẫn, thụt lề YAML; xem tab Actions
Không thấy tab Settings	Không có quyền quản trị repo	Nhờ owner cấp quyền
Không giao được issue cho Copilot	Gói Free không có cloud agent, hoặc chưa bật	Kiểm tra gói, bật cloud agent trong cài đặt Copilot (24.7)
Commit hiện sai tên, không có ảnh đại diện	Email trong `git config` không khớp email tài khoản	Đặt lại `git config --global user.email` bằng email đã thêm vào GitHub
Mất điện thoại 2FA	Không còn mã TOTP	Dùng mã khôi phục, passkey hoặc phương thức dự phòng; xem trang khôi phục tài khoản

Nguồn đối chiếu: [65][66][48][118][24][87][89][19].

HỎI AI KHI GẶP LỖI — COPILOT / GROK / CHATGPT

Tôi chạy lệnh `<lệnh>` và nhận lỗi: `<thông báo lỗi>`. Tôi dùng <Windows/macOS>, Git <phiên bản>. Giải thích nguyên nhân và cách sửa từng bước, ưu tiên cách an toàn không mất dữ liệu.

PHỤ LỤC A

Câu hỏi thường gặp

GitHub có miễn phí không?

Có. Gói Free cho repo public và private không giới hạn, 2.000 phút Actions/tháng cho repo private, Pages cho repo public[21].

Không biết code có dùng được không?

Được. Chương 1–15, 21, 26 làm hoàn toàn trên web: viết tài liệu, quản lý việc, đăng trang web.

Git và GitHub khác nhau thế nào?

Git là phần mềm quản lý phiên bản trên máy; GitHub là dịch vụ web lưu repo Git và cộng tác (1.1).

Repo private có thật sự riêng tư?

Chỉ bạn và người được mời xem được. Dù vậy, đừng lưu mật khẩu, khóa thật trong repo.

Xóa repo có khôi phục được không?

⚠ CHƯA CHẮC Docs có hướng dẫn khôi phục repo đã xóa trong một khoảng thời gian giới hạn; đừng trông chờ vào đó, hãy cân nhắc **Archive** thay vì xóa.

Đổi username có ảnh hưởng gì?

Đường dẫn cũ tới hồ sơ, repo có thể hỏng; người khác phải cập nhật link. Chọn tên cẩn thận từ đầu.

Bao nhiêu người cộng tác được trên repo Free?

Không giới hạn cộng tác viên trên repo public và private của gói Free[21].

Tải file lớn được không?

Qua web tối đa 25 MiB/file; qua dòng lệnh tối đa 100 MiB; lớn hơn dùng Git LFS[24].

Commit thẳng vào main có sao không?

Làm một mình thì được. Làm nhóm nên dùng nhánh và PR.

Fork và clone khác gì?

Fork tạo bản sao trên GitHub dưới tài khoản bạn; clone tải repo về máy.

Pull request có phải là "kéo code về"?

Không. PR là đề nghị người khác "kéo" (gộp) thay đổi của bạn vào nhánh của họ.

Lỡ merge nhầm PR?

Bấm **Revert** trên PR đã merge để tạo PR đảo ngược[34].

Có cần cài Git nếu dùng GitHub Desktop?

GitHub Desktop tự xử lý phần Git cho các thao tác thường dùng; muốn gõ lệnh thì cài Git.

Tại sao push đòi "token"?

GitHub không nhận mật khẩu tài khoản khi push qua HTTPS; dùng gh, Git Credential Manager hoặc SSH[65].

Copilot Free có đủ dùng?

Đủ để học: chat với hạn mức credits, 2.000 lượt gợi ý code/tháng. Không có cloud agent[47].

AI credits là gì?

Đơn vị tính mức dùng Copilot từ 01/06/2026; 1 credit = 0,01 USD; tùy model và lượng token[100].

Sinh viên được gì?

Sau khi xác minh qua GitHub Education: Copilot Student miễn phí; GitHub đánh giá lại tư cách hằng tháng[4].

GitHub Pages có làm web bán hàng được không?

Không nên và không được phép dùng cho thương mại điện tử, SaaS[87].

Codespaces có tốn tiền?

Tài khoản cá nhân có hạn mức miễn phí mỗi tháng; nhớ dừng, xóa codespace không dùng[93].

Actions có tốn tiền?

Repo public với runner chuẩn: miễn phí. Repo private: dùng hạn mức phút theo gói[45].

Secret gist có bí mật không?

Không hẳn: ai có link đều xem được[121].

Bị yêu cầu bật 2FA, có bắt buộc không?

Nếu tài khoản thuộc diện bắt buộc: có 45 ngày + 7 ngày ân hạn, sau đó sẽ bị chặn truy cập tới khi bật[14].

Dùng GitHub trên điện thoại được không?

Được, với ứng dụng GitHub Mobile cho iOS, Android[127].

Tôi nên bắt đầu học từ đâu?

Làm lần lượt bài tập các chương 2, 3, 4, 5, 9, 21. Sau một tuần bạn đã có hồ sơ, repo, PR và trang web.

Sách có còn đúng sau này?

Giao diện và giá có thể đổi. Các mục có nhãn "Thay đổi 2026" và "Chưa chắc" nên được kiểm tra lại trên docs.

PHỤ LỤC B

Thuật ngữ

Định nghĩa chính thức đầy đủ: GitHub glossary[3].

Thuật ngữ	Nghĩa
Action	Gói bước tự động dựng sẵn dùng trong workflow
Actions	Nền tảng tự động hóa CI/CD của GitHub
AI credits	Đơn vị tính mức dùng Copilot, 1 credit = 0,01 USD
Assignee	Người được giao issue, PR
Base branch	Nhánh đích mà PR muốn gộp vào
Blame	Xem từng dòng do commit nào tạo
Branch	Nhánh: đường làm việc song song
Checks	Kết quả kiểm thử tự động hiển thị trên PR
Clone	Tải bản sao repo về máy
Codespace	Môi trường lập trình trên mây
Commit	Một lần lưu thay đổi có lời nhắn
Compare branch	Nhánh chứa thay đổi trong PR
Conflict	Xung đột khi hai nhánh sửa cùng chỗ
Copilot cloud agent	Agent AI làm việc trên mây, tạo nhánh, commit, PR (trước gọi là coding agent)
Default branch	Nhánh mặc định, thường là main
Dependabot	Công cụ cảnh báo, cập nhật thư viện
Diff	Phần khác biệt giữa hai phiên bản
Discussion	Chủ đề trao đổi mở trong repo
Draft PR	PR nháp, chưa sẵn sàng merge
Fetch	Tải thông tin mới từ remote, chưa gộp
Fork	Bản sao repo người khác dưới tài khoản bạn
Gist	Đoạn văn bản, code chia sẻ nhanh
.gitignore	File liệt kê những gì Git bỏ qua
HEAD	Commit bạn đang đứng
Issue	Phiếu việc: lỗi, ý tưởng, nhiệm vụ
Label	Nhãn phân loại issue, PR
License	Giấy phép sử dụng dự án
Markdown	Cú pháp định dạng văn bản
Merge	Gộp thay đổi giữa các nhánh
Milestone	Mốc gom việc theo mục tiêu có hạn

Thuật ngữ	Nghĩa
Organization	Tài khoản dùng chung cho nhóm
Origin	Tên mặc định của repo GitHub mà repo máy kết nối
Passkey	Khóa đăng nhập không cần mật khẩu, chống lừa đảo
PAT	Personal access token: chuỗi thay mật khẩu cho công cụ
Project	Bảng kế hoạch gắn với issue, PR
Pull	Kéo thay đổi mới từ GitHub về máy
Pull request	Đề nghị gộp thay đổi
Push	Đẩy commit từ máy lên GitHub
Push protection	Chặn push có chứa khóa bí mật
Rebase	Đặt lại commit lên đầu nhánh khác
Release	Phiên bản đóng gói có ghi chú, file tải
Remote	Bản repo ở nơi khác (thường là GitHub)
Repository	Kho chứa file và lịch sử dự án
Review	Duyệt, góp ý thay đổi trong PR
Ruleset	Bộ luật bảo vệ nhánh, tag
Runner	Máy chạy job của Actions
Secret scanning	Quét tìm khóa bị lộ
SHA	Mã định danh duy nhất của commit
Squash	Gộp nhiều commit thành một
SSH key	Cặp khóa để kết nối Git an toàn
Staging area	Vùng chờ trước khi commit
Star	Gắn sao lưu repo, cảm ơn tác giả
Tag	Nhãn đánh dấu một điểm trong lịch sử
Team	Nhóm thành viên trong tổ chức
2FA	Xác thực hai lớp
Upstream	Repo gốc của một fork
Watch	Theo dõi thông báo của repo
Workflow	Quy trình tự động viết bằng YAML
YAML	Định dạng file cấu hình của Actions

PHỤ LỤC C

Checklist tự đánh giá

Tài khoản và bảo mật

- [ ] Có username đẹp, ảnh, bio
- [ ] Profile README hiển thị
- [ ] Bật 2FA bằng TOTP
- [ ] Thêm passkey
- [ ] Cất mã khôi phục ở 2 nơi
- [ ] Push protection for yourself đang bật

Repo và nội dung

- [ ] Tạo repo có README, license, .gitignore
- [ ] Viết Markdown: tiêu đề, bảng, task list, alert
- [ ] Tạo, tải lên, sửa, xóa file trên web
- [ ] Đọc lịch sử commit, diff, blame

Cộng tác

- [ ] Tạo nhánh, mở PR có mô tả đầy đủ
- [ ] Review với gợi ý sửa
- [ ] Merge và xóa nhánh
- [ ] Tự gây và giải một xung đột
- [ ] Issue tự đóng nhờ Closes #số
- [ ] Dùng nhãn, milestone, Projects board
- [ ] Fork và mở PR về repo gốc

Trên máy tính

- [ ] Cài Git, đặt user.name, user.email
- [ ] Đăng nhập bằng gh hoặc Credential Manager
- [ ] Clone, commit, push, pull
- [ ] Dùng GitHub Desktop commit và push
- [ ] Tạo PR bằng gh pr create

Tính năng mở rộng

- [ ] Tạo release v1.0.0
- [ ] Trang <username>.github.io hoạt động
- [ ] Workflow Actions chạy xanh
- [ ] Mở và dừng một codespace
- [ ] Dùng Copilot Chat giải thích một file
- [ ] Bật Dependabot, thêm SECURITY.md
- [ ] Inbox thông báo gọn gàng

PHỤ LỤC D

Bảng tra nhanh

GIT

```
git config --global user.name "Ten"
git config --global user.email "<email>"
git clone <URL>
git status
git add .
git commit -m "Lời nhắń"
git push  /  git push -u origin <nhanh>
git pull
git switch -c <nhanh-moi>
git switch main
git merge <nhanh>
git log --oneline
git diff
git restore <file>
git revert <SHA>
git rm --cached <file>
git tag v1.0.0 && git push origin v1.0.0
git remote -v
```

GITHUB CLI

```
gh auth login  /  gh auth status
gh status
gh repo create
gh repo clone OWNER/REPO
gh repo view --web
gh issue list  /  gh issue create
gh pr create  /  gh pr list
gh pr checkout <sôś   ⚠ xem --help
gh codespace create  /  list
gh <lệnh> --help
```

CÀI COPILOT CLI

```
npm install -g @github/copilot
winget install GitHub.Copilot
brew install --cask copilot-cli
```

SSH

```
ssh-keygen -t ed25519 -C "<email>"
ssh -T git@github.com
```

Markdown

`# Tiêu đề` · `**đậm**` · `- [ ] việc` · `[chữ](link)` · `> [!TIP]`

Phím tắt web

`t` tìm file · `.` mở github.dev · `/` tìm kiếm · `?` xem phím tắt

Nguồn lệnh: [63][69][70][71][79][72][76][106][67][31]. Lệnh có ⚠ chưa đối chiếu trực tiếp trong docs đã tải.

D.2 Prompt mẫu cho Copilot Chat (hoặc trợ lý AI bất kỳ)

Mục đích	Prompt
Hiểu repo	"Repo này làm gì? Giải thích cấu trúc thư mục cho người mới, bằng tiếng Việt."
Hiểu file	"Giải thích file <tên-file> từng phần, kèm ví dụ."
Viết README	"Viết README tiếng Việt cho repo này: giới thiệu, tính năng, cách dùng, cách đóng góp, license."
Viết issue	"Viết issue báo lỗi: <mô tả>. Gồm các bước tái hiện, kết quả mong đợi, kết quả thực tế."
Mô tả PR	"Tóm tắt thay đổi của PR này thành: Thay đổi, Lý do, Cách kiểm tra."
Review	"Review thay đổi này, chỉ ra lỗi tiềm ẩn, vấn đề bảo mật, đề xuất cải thiện."
Lời nhắn commit	"Đề xuất lời nhắn commit ngắn, rõ cho các thay đổi đã stage."
Xung đột	"Giải thích xung đột merge sau và gợi ý cách giữ cả hai thay đổi: <đoạn xung đột>."
.gitignore	"Tạo .gitignore cho dự án <Node.js/Python/web tĩnh>, có mục cho file .env."
Actions	"Viết workflow GitHub Actions chạy khi push, kiểm tra liên kết hỏng trong các file .md."
Pages	"Tạo index.html một trang giới thiệu cá nhân, đơn giản, đẹp trên điện thoại, cho GitHub Pages."
Lỗi Git	"Tôi gặp lỗi <thông báo> khi <lệnh>. Giải thích và sửa từng bước, không làm mất dữ liệu."
Giao việc cho agent	"Thêm trang Liên hệ; hoàn thành khi: <tiêu chí>; không sửa: <phạm vi>."
Học tập	"Ra cho tôi 5 câu hỏi trắc nghiệm về branch và pull request, có đáp án và giải thích."

NGUYÊN TẮC VIẾT PROMPT

- Nêu **bối cảnh** (repo, file), **mục tiêu**, **ràng buộc**, **định dạng** đầu ra.
- Không dán khóa, token, dữ liệu cá nhân; thay bằng `<YOUR_API_KEY>`.
- Luôn đọc lại và kiểm tra kết quả trước khi commit.

PHỤ LỤC E

Danh mục video

Lượt xem đo bằng yt-dlp (chỉ siêu dữ liệu, không tải video) ngày 17/10/2026. Đoạn chọn theo chương (chapters) của video, mỗi đoạn 1–5 phút. Mọi mã QR đã được giải mã thử bằng zbarimg và khớp đúng liên kết `youtu.be/<id>?t=<giây>`. Video do kênh sở hữu; sách chỉ dẫn link.

Video	Kênh	Lượt xem	Đoạn xem	Dài
Git Explained in 100 Seconds	Fireship	841.845	00:00–01:57	1:57
A brief introduction to Git for beginners | GitHub	GitHub	2.225.973	00:44–03:06	2:22
A brief introduction to Git for beginners | GitHub	GitHub	2.225.973	05:10–08:29	3:19
Git and GitHub Tutorial for Beginners	Kevin Stratvert	2.023.896	35:09–37:29	2:20
Git and GitHub Tutorial for Beginners	Kevin Stratvert	2.023.896	09:18–11:23	2:05
Git and GitHub Tutorial for Beginners	Kevin Stratvert	2.023.896	23:12–27:19	4:07
Git and GitHub for Beginners - Crash Course	freeCodeCamp.org	5.160.077	14:30–17:30	3:00
Git, GitHub, & GitHub Desktop for beginners	Coder Coder	1.038.533	02:48–04:12	1:24
How to create a pull request in 4 min | GitHub for Beginners	GitHub	331.685	00:59–03:44	2:45
Git & GitHub Tutorial for Beginners #9 - Merging Branches (& conflicts)	Net Ninja	501.915	03:38–08:20	4:42
How to use GitHub issues and projects | GitHub for Beginners	GitHub	69.263	01:15–05:20	4:05
Getting started with GitHub Pages for beginners | Tutorial	GitHub	48.218	00:55–03:31	2:36
How to use GitHub Actions | GitHub for Beginners	GitHub	62.395	02:31–07:26	4:55
How to use GitHub Codespaces for Coding and Data Science	Data Professor	77.968	03:17–06:08	2:51
How to use GitHub Copilot (the complete beginner's guide)	GitHub	195.970	05:35–08:14	2:39
How the GitHub Copilot coding agent works | GitHub Checkout	GitHub	75.042	00:37–02:36	1:59
Getting started with GitHub security | GitHub for Beginners	GitHub	28.871	02:36–04:37	2:01

LƯU Ý

Video ghi hình ở thời điểm cũ hơn sách có thể khác giao diện và giá hiện tại; ví dụ video Copilot (07/2025) nói về gói cước trước khi chuyển sang AI credits, video agent (05/2025) dùng tên “coding agent”. Ưu tiên nội dung trong sách và docs khi có khác biệt.

PHỤ LỤC F

Nguồn tham khảo

Truy cập ngày 16–17/10/2026 (UTC+7). Số trong ngoặc vuông tương ứng chỉ số trích dẫn trong sách.

[1] What is GitHub?
docs.github.com/en/get-started/start-your-journey/about-github-and-git

[2] About Git
docs.github.com/en/get-started/using-git/about-git

[3] GitHub glossary
docs.github.com/en/get-started/learning-about-github/github-glossary

[4] Access GitHub Copilot for free as a student
docs.github.com/en/copilot/how-tos/copilot-on-github/set-up-copilot/enable-copilot/set-up-for-students

[5] GitHub flow
docs.github.com/en/get-started/using-github/github-flow

[6] Hello World
docs.github.com/en/get-started/start-your-journey/hello-world

[7] Types of GitHub accounts
docs.github.com/en/get-started/learning-about-github/types-of-github-accounts

[8] About organizations
docs.github.com/en/organizations/collaborating-with-groups-in-organizations/about-organizations

[9] Creating an account on GitHub
docs.github.com/en/get-started/start-your-journey/creating-an-account-on-github

[10] About your profile
docs.github.com/en/account-and-profile/setting-up-and-managing-your-github-profile/customizing-your-profile/about-your-profile

[11] Managing your profile README
docs.github.com/en/account-and-profile/setting-up-and-managing-your-github-profile/customizing-your-profile/managing-your-profile-readme

[12] Quickstart for repositories
docs.github.com/en/repositories/creating-and-managing-repositories/quickstart-for-repositories

[13] About two-factor authentication
docs.github.com/en/authentication/securing-your-account-with-two-factor-authentication-2fa/about-two-factor-authentication

[14] About mandatory two-factor authentication
docs.github.com/en/authentication/securing-your-account-with-two-factor-authentication-2fa/about-mandatory-two-factor-authentication

[15] Configuring two-factor authentication
docs.github.com/en/authentication/securing-your-account-with-two-factor-authentication-2fa/configuring-two-factor-authentication

[16] About passkeys
docs.github.com/en/authentication/authenticating-with-a-passkey/about-passkeys

[17] Managing your passkeys
docs.github.com/en/authentication/authenticating-with-a-passkey/managing-your-passkeys

[18] Configuring two-factor authentication recovery methods
docs.github.com/en/authentication/securing-your-account-with-two-factor-authentication-2fa/configuring-two-factor-authentication-recovery-methods

[19] Recovering your account if you lose your 2FA credentials
docs.github.com/en/authentication/securing-your-account-with-two-factor-authentication-2fa/recovering-your-account-if-you-lose-your-2fa-credentials

[20] Setting repository visibility
docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/managing-repository-settings/setting-repository-visibility

[21] GitHub's plans
docs.github.com/en/get-started/learning-about-github/githubs-plans

[22] About repositories
docs.github.com/en/repositories/creating-and-managing-repositories/about-repositories

[23] Inviting collaborators to a personal repository
docs.github.com/en/account-and-profile/setting-up-and-managing-your-personal-account-on-github/managing-access-to-your-personal-repositories/inviting-collaborators-to-a-personal-repository

[24] About large files on GitHub
docs.github.com/en/repositories/working-with-files/managing-large-files/about-large-files-on-github

[25] About the repository README file
docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-readmes

[26] Basic writing and formatting syntax
docs.github.com/en/get-started/writing-on-github/getting-started-with-writing-and-formatting-on-github/basic-writing-and-formatting-syntax

[27] Autolinked references and URLs
docs.github.com/en/get-started/writing-on-github/working-with-advanced-formatting/autolinked-references-and-urls

[28] Creating new files
docs.github.com/en/repositories/working-with-files/managing-files/creating-new-files

[29] Adding a file to a repository
docs.github.com/en/repositories/working-with-files/managing-files/adding-a-file-to-a-repository

[30] Editing files
docs.github.com/en/repositories/working-with-files/managing-files/editing-files

[31] Keyboard shortcuts
docs.github.com/en/get-started/accessibility/keyboard-shortcuts

[32] Commits
docs.github.com/en/pull-requests/committing-changes-to-your-project/creating-and-editing-commits/about-commits

[33] Creating a pull request
docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/creating-a-pull-request

[34] Merging a pull request
docs.github.com/en/pull-requests/collaborating-with-pull-requests/incorporating-changes-from-a-pull-request/merging-a-pull-request

[35] Removing sensitive data from a repository
docs.github.com/en/authentication/keeping-your-account-and-data-secure/removing-sensitive-data-from-a-repository

[36] Branches
docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-branches

[37] About protected branches
docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches

[38] About rulesets
docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-rulesets/about-rulesets

[39] Pull requests
docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-pull-requests

[40] About issues
docs.github.com/en/issues/tracking-your-work-with-issues/about-issues

[41] Changelog 06/10/2026: Stacked pull requests generally available
github.blog/changelog/2026-10-06-stacked-pull-requests-generally-available

[42] Resolving a merge conflict on GitHub
docs.github.com/en/pull-requests/collaborating-with-pull-requests/addressing-merge-conflicts/resolving-a-merge-conflict-on-github

[43] Pull request reviews
docs.github.com/en/pull-requests/collaborating-with-pull-requests/reviewing-changes-in-pull-requests/about-pull-request-reviews

[44] About merge methods on GitHub
docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/configuring-pull-request-merges/about-merge-methods-on-github

[45] GitHub Actions billing
docs.github.com/en/billing/concepts/product-billing/github-actions

[46] Changelog 01/06/2026: Updates to GitHub Copilot billing and plans
github.blog/changelog/2026-06-01-updates-to-github-copilot-billing-and-plans/

[47] Plans for GitHub Copilot
docs.github.com/en/copilot/get-started/plans

[48] Merge conflicts
docs.github.com/en/pull-requests/collaborating-with-pull-requests/addressing-merge-conflicts/about-merge-conflicts

[49] Resolving a merge conflict using the command line
docs.github.com/en/pull-requests/collaborating-with-pull-requests/addressing-merge-conflicts/resolving-a-merge-conflict-using-the-command-line

[50] Quickstart for GitHub Issues
docs.github.com/en/issues/tracking-your-work-with-issues/quickstart

[51] Managing labels
docs.github.com/en/issues/using-labels-and-milestones-to-track-work/managing-labels

[52] About milestones
docs.github.com/en/issues/using-labels-and-milestones-to-track-work/about-milestones

[53] About Projects
docs.github.com/en/issues/planning-and-tracking-with-projects/learning-about-projects/about-projects

[54] Quickstart for Projects
docs.github.com/en/issues/planning-and-tracking-with-projects/learning-about-projects/quickstart-for-projects

[55] Quickstart for GitHub Discussions
docs.github.com/en/discussions/quickstart

[56] About discussions
docs.github.com/en/discussions/collaborating-with-your-community-using-discussions/about-discussions

[57] Saving repositories with stars
docs.github.com/en/get-started/exploring-projects-on-github/saving-repositories-with-stars

[58] About notifications
docs.github.com/en/account-and-profile/managing-subscriptions-and-notifications-on-github/setting-up-notifications/about-notifications

[59] Viewing your subscriptions
docs.github.com/en/account-and-profile/managing-subscriptions-and-notifications-on-github/managing-subscriptions-for-activity-on-github/viewing-your-subscriptions

[60] Following people
docs.github.com/en/get-started/exploring-projects-on-github/following-people

[61] Forks
docs.github.com/en/pull-requests/collaborating-with-pull-requests/working-with-forks/about-forks

[62] Contributing to a project
docs.github.com/en/get-started/exploring-projects-on-github/contributing-to-a-project

[63] Set up Git
docs.github.com/en/get-started/git-basics/set-up-git

[64] Setting your username in Git
docs.github.com/en/get-started/git-basics/setting-your-username-in-git

[65] Caching your GitHub credentials in Git
docs.github.com/en/get-started/git-basics/caching-your-github-credentials-in-git

[66] About SSH
docs.github.com/en/authentication/connecting-to-github-with-ssh/about-ssh

[67] Generating a new SSH key and adding it to the ssh-agent
docs.github.com/en/authentication/connecting-to-github-with-ssh/generating-a-new-ssh-key-and-adding-it-to-the-ssh-agent

[68] Managing your personal access tokens
docs.github.com/en/authentication/keeping-your-account-and-data-secure/managing-your-personal-access-tokens

[69] Cloning a repository
docs.github.com/en/repositories/creating-and-managing-repositories/cloning-a-repository

[70] Pushing commits to a remote repository
docs.github.com/en/get-started/using-git/pushing-commits-to-a-remote-repository

[71] Getting changes from a remote repository
docs.github.com/en/get-started/using-git/getting-changes-from-a-remote-repository

[72] GitHub CLI quickstart
docs.github.com/en/github-cli/github-cli/quickstart

[73] Installing GitHub Desktop
docs.github.com/en/desktop/installing-and-authenticating-to-github-desktop/installing-github-desktop

[74] Getting started with GitHub Desktop
docs.github.com/en/desktop/overview/getting-started-with-github-desktop

[75] Creating your first repository using GitHub Desktop
docs.github.com/en/desktop/overview/creating-your-first-repository-using-github-desktop

[76] GitHub CLI: hướng dẫn cài đặt (repo cli/cli, docs/install_*.md)
github.com/cli/cli#installation

[77] About GitHub CLI
docs.github.com/en/github-cli/github-cli/about-github-cli

[78] GitHub CLI reference
docs.github.com/en/github-cli/github-cli/github-cli-reference

[79] Ignoring files
docs.github.com/en/get-started/git-basics/ignoring-files

[80] Licensing a repository
docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/licensing-a-repository

[81] About releases
docs.github.com/en/repositories/releasing-projects-on-github/about-releases

[82] Managing releases in a repository
docs.github.com/en/repositories/releasing-projects-on-github/managing-releases-in-a-repository

[83] Viewing your repository's releases and tags
docs.github.com/en/repositories/releasing-projects-on-github/viewing-your-repositorys-releases-and-tags

[84] What is GitHub Pages?
docs.github.com/en/pages/getting-started-with-github-pages/what-is-github-pages

[85] Quickstart for GitHub Pages
docs.github.com/en/pages/quickstart

[86] Configuring a publishing source for your GitHub Pages site
docs.github.com/en/pages/getting-started-with-github-pages/configuring-a-publishing-source-for-your-github-pages-site

[87] GitHub Pages limits
docs.github.com/en/pages/getting-started-with-github-pages/github-pages-limits

[88] Understanding GitHub Actions
docs.github.com/en/actions/get-started/understand-github-actions

[89] Quickstart for GitHub Actions
docs.github.com/en/actions/get-started/quickstart

[90] Secrets
docs.github.com/en/actions/concepts/security/secrets

[91] What are GitHub Codespaces?
docs.github.com/en/codespaces/about-codespaces/what-are-codespaces

[92] Introduction to dev containers
docs.github.com/en/codespaces/setting-up-your-project-for-codespaces/adding-a-dev-container-configuration/introduction-to-dev-containers

[93] GitHub Codespaces billing
docs.github.com/en/billing/concepts/product-billing/github-codespaces

[94] Pricing · Plans for every developer (github.com/pricing)
github.com/pricing

[95] Creating a codespace for a repository
docs.github.com/en/codespaces/developing-in-a-codespace/creating-a-codespace-for-a-repository

[96] Quickstart for GitHub Codespaces
docs.github.com/en/codespaces/quickstart

[97] Stopping and starting a codespace
docs.github.com/en/codespaces/developing-in-a-codespace/stopping-and-starting-a-codespace

[98] Where to use GitHub Copilot
docs.github.com/en/copilot/get-started/where-to-use-github-copilot

[99] GitHub Copilot on GitHub.com
docs.github.com/en/copilot/concepts/agents/cloud-agent/about-cloud-agent

[100] Usage-based billing for individuals
docs.github.com/en/copilot/concepts/billing-and-usage/individuals/billing

[101] Usage-based billing for organizations and enterprises
docs.github.com/en/enterprise-cloud@latest/copilot/concepts/billing/usage-based-billing-for-organizations-and-enterprises

[102] Blog 27/04/2026: GitHub Copilot is moving to usage-based billing
github.blog/news-insights/company-news/github-copilot-is-moving-to-usage-based-billing/

[103] Quickstart for using GitHub Copilot on GitHub.com
docs.github.com/en/copilot/get-started/quickstart

[104] Asking GitHub Copilot questions in GitHub
docs.github.com/en/copilot/how-tos/chat-with-copilot/chat-in-github

[105] GitHub Copilot CLI
docs.github.com/en/copilot/concepts/agents/about-copilot-cli

[106] Installing GitHub Copilot CLI
docs.github.com/en/copilot/how-tos/set-up/install-copilot-cli

[107] Changelog 17/06/2026: GitHub Copilot app generally available
github.blog/changelog/2026-06-17-github-copilot-app-generally-available/

[108] Changelog 01/10/2026: GitHub Copilot can now interact with desktop apps with computer use
github.blog/changelog/2026-10-01-github-copilot-can-now-interact-with-desktop-apps/

[109] Changelog 07/10/2026: Local sandboxing for GitHub Copilot now generally available
github.blog/changelog/2026-10-07-local-sandboxing-for-github-copilot-now-generally-available

[110] Changelog 09/10/2026: GitHub Copilot weekly releases (October 5)
github.blog/changelog/2026-10-09-github-copilot-weekly-releases-october-5/

[111] GitHub Copilot code review
docs.github.com/en/copilot/concepts/agents/code-review

[112] GitHub security features
docs.github.com/en/code-security/getting-started/github-security-features

[113] Dependabot alerts
docs.github.com/en/code-security/dependabot/dependabot-alerts/about-dependabot-alerts

[114] Dependabot version updates
docs.github.com/en/code-security/dependabot/dependabot-version-updates/about-dependabot-version-updates

[115] Secret scanning
docs.github.com/en/code-security/secret-scanning/introduction/about-secret-scanning

[116] Code scanning
docs.github.com/en/code-security/code-scanning/introduction-to-code-scanning/about-code-scanning

[117] Managing push protection for users
docs.github.com/en/code-security/secret-scanning/working-with-secret-scanning-and-push-protection/push-protection-for-users

[118] Push protection
docs.github.com/en/code-security/secret-scanning/introduction/about-push-protection

[119] Changelog 13/10/2026: Secret scanning blocks Anthropic keys by default, adds Salesforce
github.blog/changelog/2026-10-13-secret-scanning-blocks-anthropic-keys-by-default-adds-salesforce

[120] Adding a security policy to your repository
docs.github.com/en/code-security/getting-started/adding-a-security-policy-to-your-repository

[121] Creating gists
docs.github.com/en/get-started/writing-on-github/editing-and-sharing-content-with-gists/creating-gists

[122] Creating a new organization from scratch
docs.github.com/en/organizations/collaborating-with-groups-in-organizations/creating-a-new-organization-from-scratch

[123] About organization teams
docs.github.com/en/organizations/organizing-members-into-teams/about-teams

[124] Managing notifications from your inbox
docs.github.com/en/account-and-profile/managing-subscriptions-and-notifications-on-github/viewing-and-triaging-notifications/managing-notifications-from-your-inbox

[125] Changelog 01/10/2026: New dashboard experience now the default
github.blog/changelog/2026-10-01-new-dashboard-experience-now-the-default

[126] Changelog 15/10/2026: Email notifications for repository push events have been retired
github.blog/changelog/2026-10-15-email-notifications-for-repository-push-events-have-been-retired

[127] GitHub Mobile
docs.github.com/en/get-started/using-github/github-mobile

Cảm ơn bạn đã đọc tới trang cuối

Hẹn gặp bạn trên GitHub

Theo dõi Minh để nhận thêm hướng dẫn AI và công cụ số dễ hiểu. Tham gia nhóm Zalo để hỏi đáp, chia sẻ repo và trang web đầu tiên của bạn.

Facebook
facebook.com/dotanminh

YouTube
youtube.com/@minhdovuive

Nhóm Zalo
zalo.me/g/yriwgtmp848prmxbhdv1

Minh Đỗ

AI EDUCATION CREATOR

BẢN QUYỀN

© 2026 Minh Đỗ. Ebook "Hướng dẫn sử dụng GitHub từ A đến Z, toàn tập". Chia sẻ miễn phí kèm ghi nguồn; không bán lại, không chỉnh sửa khi chưa có sự đồng ý. Tài liệu độc lập, không liên kết, không được GitHub bảo trợ. GitHub, Copilot là nhãn hiệu của GitHub, Inc. Ảnh giao diện thuộc GitHub Docs; video thuộc các kênh sở hữu. Thông tin cập nhật tới 17/10/2026; giao diện, giá có thể thay đổi.